
GraphFLEx: Unsupervised Structure Learning Framework for Large Expanding Graphs

Mohit Kataria¹ Nikita Malik² Jayadeva^{1,3} Sandeep Kumar^{1,2,3}

Abstract

Graph structure learning is a core problem in graph-based machine learning, essential for uncovering latent relationships and ensuring model interpretability. However, most existing approaches are ill-suited for large-scale and dynamically evolving graphs, as they often require complete re-learning of the structure upon the arrival of new nodes and incur substantial computational and memory costs. In this work, we propose GraphFLEx—a unified and scalable framework for Graph Structure Learning in Large and Expanding Graphs. GraphFLEx mitigates the scalability bottlenecks by restricting edge formation to structurally relevant subsets of nodes identified through a combination of clustering and coarsening techniques. This dramatically reduces the search space and enables efficient, incremental graph updates. The framework supports 48 flexible configurations by integrating diverse choices of learning paradigms, coarsening strategies, and clustering methods, making it adaptable to a wide range of graph settings and learning objectives. Extensive experiments across 26 diverse datasets and graph neural network architectures demonstrate that GraphFLEx achieves state-of-the-art performance with significantly improved scalability. Our implementation is publicly available [here](#).

1 Introduction

Graph representations capture relationships between entities, vital across diverse fields like biology, finance, sociology, engineering, and operations research (Zhou et al., 2020; Fout et al., 2017; Wu et al., 2020; Malik et al., 2025). While some relationships, such as social connections or sensor networks, are directly observable, many, including gene regulatory networks, scene graph generation (Gu et al., 2019), brain networks, (Zhu et al., 2021) and drug interactions, require inference (Allen et al., 2012). Even when available, graph data often contains noise, requiring denoising and recalibration. In such cases, inferring the correct graph structure becomes more crucial than the specific graph model or downstream algorithm. *Graph Structure Learning (GSL)* offers a solution, enabling the construction and refinement of graph topologies. GSL has been widely studied in both supervised and unsupervised contexts (Liu et al., 2022b; Chen & Wu, 2022). In supervised GSL (s-GSL), the adjacency matrix and Graph Neural Networks (GNNs) are jointly optimized for a downstream task, such as node classification. Notable examples of s-GSL include *NodeFormer* (Wu et al., 2022b), *Pro – GNN* (Jin et al., 2020), *WSGNN* (Lao et al., 2022), and *SLAPS* (Fatemi et al., 2021). Unsupervised GSL (u-GSL), on the other hand, focuses solely on learning the underlying graph structure, typically through adjacency or laplacian matrices. Methods in this category include approximate nearest neighbours ($A - NN$) (Dong et al., 2011; Muja & Lowe, 2014), k -nearest neighbours ($k - NN$) (MacQueen et al., 1967; Wang & Zhang, 2006), covariance estimation (*emp.Cov.*) (Hsieh et al., 2011), graphical lasso (*GLasso*) (Friedman et al., 2008), *SUBLIME* (Liu et al., 2022b), and signal processing techniques like l_2 -model, log -model and *large*-model (Dong et al., 2016; Kalofolias, 2016).

Supervised graph structure learning (s-GSL) methods have demonstrated effectiveness in specific tasks; however, their reliance on labeled data and optimization for downstream objectives particularly node classification, significantly constrains their generalizability to settings where annotations are scarce or unavailable (Liu et al., 2022b). Unsupervised graph structure learning (u-GSL) methods, which constitute the focus of this work, offer broader applicability. Never-

¹Yardi School of Artificial Intelligence, Indian Institute of Technology Delhi, India ²Bharti School of Telecommunication Technology and Management, Indian Institute of Technology Delhi, India ³Department of Electrical Engineering, Indian Institute of Technology Delhi, India. Correspondence to: Mohit Kataria <Mohit.Kataria@scai.iitd.ac.in>.

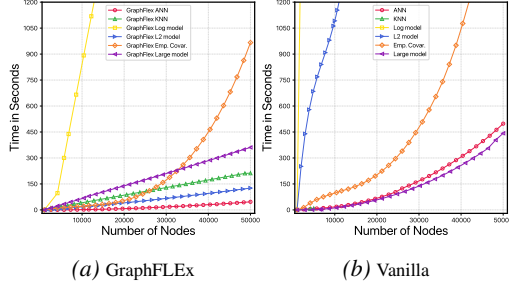


Figure 1. High computational time required to learn graph structures using existing methods, whereas GraphFLEX effectively controls computational growth, achieving near-linear scalability. Notably, Vanilla KNN failed to construct graph structures for more than 10K nodes due to memory limitations.

theless, both s-GSL and u-GSL approaches exhibit critical limitations in their ability to scale to large graphs or adapt efficiently to expanding datasets. To address these challenges, we introduce GRAPHFLEX, a unified and scalable framework for *Graph Structure Learning in Large and Expanding Graphs*. GraphFLEX is built upon the coordinated integration of three foundational paradigms in graph processing: *graph clustering*, *graph coarsening*, and *structure learning*.

GraphFLEX uses these components in the principled manner in which they are algorithmically aligned to reinforce one another; clustering serves to localize the search space, coarsening reduces structural redundancy while preserving global properties, and structure learning operates efficiently within this refined context. This integration enables GraphFLEX to scale effectively to large datasets and accommodate dynamic graphs through *incremental updates*, eliminating the need for expensive re-training. Additionally, the framework supports **48 modular configurations**, enabling broad adaptability across datasets, learning objectives, and deployment constraints. Crucially, we establish *theoretical guarantees* on edge recovery fidelity and computational complexity, offering rigorous foundations for the framework’s efficiency and reliability. As illustrated in Figure 2, GraphFLEX significantly reduces the candidate edge space by operating on structurally relevant node subsets. Empirical evaluations, summarized in Figure 1, demonstrate that GraphFLEX substantially outperforms existing baselines in both runtime and scalability.

Scope. GraphFLEX study *unsupervised* graph structure learning (u-GSL), where edge inference must be performed *without labels*. Supervised GSL (s-GSL) methods optimize graph structure using downstream supervision and typically assume an observed base graph; they address a different problem setting and are therefore not directly comparable in our primary evaluation.

Key contributions of GraphFLEX.

- We unify clustering, coarsening, and structure learning into a single modular pipeline that supports incremental

updates for *large graphs* and *growing graphs*.

- Provides both *theoretical guarantees* and *empirical evidence*, validating performance across 26 diverse datasets.
- GraphFLEX enables unsupervised-GSL on graphs with up to ~ 2.4 M nodes, a regime where prior GSL methods are typically infeasible.
- A library of 48 modular configurations to accommodate diverse graph learning objectives and data settings.

2 Problem Formulation and Background

A graph $\mathcal{G}$ is represented using $\mathcal{G}(V, A, X)$ where $V = \{v_1, v_2, \dots, v_N\}$ is the set of N nodes, each node v_i has a d -dimensional feature vector x_i in $X \in \mathbb{R}^{N \times d}$ and $A \in \mathbb{R}^{N \times N}$ is adjacency matrix representing connection between i^{th} and j^{th} nodes when entry $A_{ij} > 0$. An expanding graph $\mathcal{E}_{\mathcal{G}}$ can be considered a variant of graph $\mathcal{G}$ where nodes v now have an associated timestamp τ_v . We can represent a expanding graph as a sequence of graphs, i.e., $\mathcal{E}_{\mathcal{G}} = \{\mathcal{G}_0, \mathcal{G}_1, \dots, \mathcal{G}_T\}$ where $\{\mathcal{G}_0 \subseteq \mathcal{G}_1 \dots \subseteq \mathcal{G}_T\}$ at $\tau \in \{0, \dots, T\}$ timestamps. New nodes arriving at different timestamps are seamlessly integrating into initial graph $\mathcal{G}_0$.

Problem statement. Given a partially known or missing graph structure, our goal is to incrementally learn the whole graph (A or laplacian matrix). Specifically, we consider two unsupervised GSL goals:

Goal 1. Large Datasets with Missing Graph Structure: Here, the graph structure is entirely unavailable, and existing methods are computationally infeasible for learning the whole graph in a single step. To address this, we first randomly partition the dataset into exclusive subsets. We then learn the initial graph $\mathcal{G}_0(V_0, X_0)$ over a small subset of nodes and incrementally expand it by integrating additional partitions, ultimately reconstructing the full graph $\mathcal{G}_T$.

Goal 2. Partially Available Graph: In this case, we only have access to the graph $\mathcal{G}_t$ at timestamp t , with new nodes arriving over time. The goal is to update the graph incrementally to obtain $\mathcal{G}_T$, without re-learning it from scratch at each timestamp.

2.1 Graph Reduction

Graph reduction encompasses sparsification, clustering, coarsening, and condensation (Hashemi et al., 2024). GraphFlex employs clustering and coarsening to refine the set of relevant nodes for potential connections.

Graph Clustering. Graphs often exhibit global heterogeneity with localized homogeneity, making them well-suited for clustering (Fortunato, 2010). Clusters capture higher-order structures, aiding graph learning. Methods like DMoN (Tsitsulin et al., 2023) use GNNs for soft cluster assignments, while Spectral Clustering (SC) (Kamvar et al.,

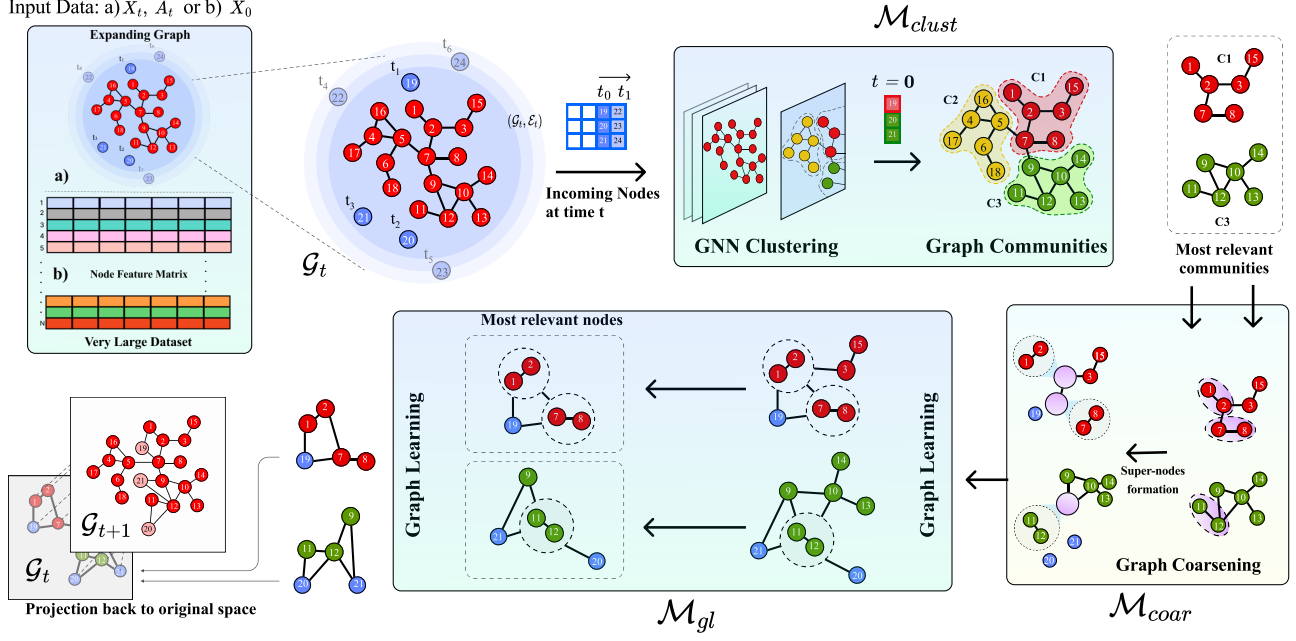


Figure 2. This figure illustrates the general pipeline of GraphFLEx, designed to efficiently handle both a) large datasets with missing structure and b) expanding graphs. Both scenarios can be modeled as expanding graphs (details in Section 3.1). GraphFLEx processes a graph ($\mathcal{G}_t$) and incoming nodes ($\mathcal{E}_{t+1}$) at time t , newly arriving nodes are shown with different timestamps and shades of blue to indicate their arrival time. Our framework comprises of three main components: i) **Clustering**, which infers $\mathcal{E}_{t+1}$ nodes to existing communities using a pre-trained model $\mathcal{M}_{clust}(\mathcal{G}_0)$ into smaller, more manageable communities; ii) Since these communities may still be large, a **Coarsening** module is applied to further reduce their size while preserving essential structural information; and iii) Finally, a **Learning** module, where the structure associated with $\mathcal{E}_{t+1}$ nodes are learned using the coarsened graph, followed by projecting this structure onto the $\mathcal{G}_t$ graph to create graph $\mathcal{G}_{t+1}$.

2003) and K-means (Wagstaff et al., 2001; MacQueen et al., 1967) efficiently detect communities. DiffPool (Bruna et al., 2014; Defferrard et al., 2016) applies SC for pooling in GNNs.

Graph Coarsening. Graph Coarsening (GC) reduces a graph $\mathcal{G}(V, E, X)$ with N nodes and features $X \in \mathbb{R}^{N \times d}$ into a smaller graph $\mathcal{G}_c(\tilde{V}, \tilde{E}, \tilde{X})$ with $n \ll N$ nodes and $\tilde{X} \in \mathbb{R}^{n \times d}$. This is achieved via learning a coarsening matrix $\mathcal{P} \in \mathbb{R}^{n \times N}$, mapping similar nodes in $\mathcal{G}$ to super-nodes in $\mathcal{G}_c$, ensuring $\tilde{X} = \mathcal{P}X$ while preserving key properties (Loukas, 2019; Kataria et al., 2023; Kumar et al., 2023; Kataria et al., 2024).

2.2 Unsupervised Graph Structure Learning

Unsupervised graph learning spans from simple k-NN weighting (Wang & Zhang, 2006; Zhu et al., 2003) to advanced statistical and graph signal processing (GSP) techniques. Statistical methods, also known as probabilistic graphical models, assume an underlying graph $\mathcal{G}$ governs the joint distribution of data $X \in \mathbb{R}^{N \times d}$ (Koller & Friedman, 2009; Banerjee et al., 2008; Friedman et al., 2008). Some approaches (Dempster, 1972) prune elements in the inverse sample covariance matrix

$\hat{\Sigma} = \frac{1}{d-1} X X^T$ and sparse inverse covariance estimators, such as Graphical Lasso (GLasso) (Friedman et al., 2008): $\max_{\Theta} \log \det \Theta - \text{tr}(\hat{\Sigma} \Theta) - \rho \|\Theta\|_1$, where Θ is the inverse covariance matrix. However, these methods struggle with small sample sizes. Graph Signal Processing (GSP) techniques analyze signals on known graphs, ensuring properties like smoothness and sparsity. Signal smoothness on a graph $\mathcal{G}$ is quantified by the Laplacian quadratic form: $Q(\mathbf{L}) = \mathbf{x}^T \mathbf{L} \mathbf{x} = \frac{1}{2} \sum_{i,j} w_{ij} (\mathbf{x}(i) - \mathbf{x}(j))^2$. For a set of vectors X , smoothness is measured using the Dirichlet energy (Belkin et al., 2006): $\text{tr}(X^T L X)$. State-of-the-art methods *l2-model*, *log-model* and *large-model* (Dong et al., 2016; Kalofolias, 2016; Hu et al., 2013) optimize Dirichlet energy while enforcing sparsity or specific structural constraints. Table 10 in Appendix F compares various graph learning methods based on their formulations and time complexities. More recently, SUBLIME (Liu et al., 2022b) learns graph structure in an unsupervised manner by leveraging self-supervised contrastive learning to align a learnable graph with a dynamically refined anchor graph derived from the data itself.

Remark 2.1. Graph Structure Learning (GSL) differs significantly from Continual Learning (CL) (Van de Ven & Tolias, 2019; Zhang et al., 2022; Parisi et al., 2019) and Dynamic

Graph Learning (DGL) (Kim et al., 2022; Wu et al., 2023; You et al., 2022), as discussed in Appendix E.

3 GraphFLEx

Here, we present GraphFLEx which has three main modules: 1) *Graph Clustering*. It identifies communities and extracts higher-order structural information, 2) *Graph Coarsening*. It is used to coarsen down the desired community, if the community itself is large, and 3) *Graph Learning*. Learns the graph’s structure using a limited subset of nodes from the clustering and coarsening modules, *enabling scalability*. For pseudocode, see Algorithm 1 in Appendix B.

3.1 Incremental Graph Learning for Large Datasets

Real-world graph data is continuously expanding. For instance, e-commerce networks accumulate new clicks and purchases daily (Xiang et al., 2010), while academic networks grow with new researchers and publications (Wang et al., 2020). To manage such growth, we propose incrementally learning the graph structure over smaller segments. Given a large dataset $\mathcal{L}(V_{\mathcal{L}}, X_{\mathcal{L}})$, where $V_{\mathcal{L}}$ is the node set and $X_{\mathcal{L}}$ represents node features, we define an *expanding dataset* setting $\mathcal{L}_{\mathcal{E}} = \{\mathcal{E}_{\tau=0}^T\}$. Initially, $\mathcal{L}$ is split into: (i) a *static dataset* $\mathcal{E}_0(V_0, X_0)$ and (ii) an *expanding dataset* $\mathcal{E} = \{\mathcal{E}_{\tau}(V_{\tau}, X_{\tau})\}_{\tau=1}^T$. Both *Goal 1* (large datasets with missing graph structure) and *Goal 2* (partially available graphs with incremental updates), discussed in Section 2, share the common objective of incrementally learning and updating the graph structure as new data arrives. GraphFLEx handles these by decomposing the problem into two key components:

- *Initial Graph* $\mathcal{G}_0(V_0, A_0, X_0)$: For *Goal 1*, where the graph structure is entirely missing, $\mathcal{E}_0(V_0, X_0)$ is used to construct $\mathcal{G}_0$ from scratch using structure learning methods (Sec.2.2). For *Goal 2*, the initial graph $\mathcal{G}_0(V_0, A_0, X_0)$ is already available and serves as the starting point for incremental updates.
- *Expanding Dataset* $\mathcal{E} = \{\mathcal{E}_{\tau}(V_{\tau}, X_{\tau})\}_{\tau=1}^T$: In both cases, $\mathcal{E}$ consists of incoming nodes and features arriving over T timestamps. These nodes are progressively integrated into the existing graph, enabling continuous adaptation and growth.

The partition is controlled by a parameter r , which determines the proportion of static nodes: $r = \frac{\|V_0\|}{\|V_{\mathcal{L}}\|}$. For example, $r = 0.2$ implies that 20% of $V_{\mathcal{L}}$ is treated as static, while the remaining 80% arrives incrementally over T timestamps. In our experiments, we set $r = 0.5$ and $T = 25$.

Remark 3.1. We can learn $\mathcal{G}_{\tau}(V_{\tau}, A_{\tau}, X_{\tau})$ by aggregating $\mathcal{E}_{\tau}$ nodes in $\mathcal{G}_{\tau-1}$ graph. We aim to learn $\mathcal{G}_T(V_T, A_T, X_T)$ after T^{th} -timestamp.

3.2 Detecting Communities

From the static graph $\mathcal{G}_0$, our goal is to learn higher-order structural information, identifying potential communities to which incoming nodes ($V \in V_{\tau}$) may belong. We train the community detection/clustering model $\mathcal{M}_{\text{clust}}$ once using $\mathcal{G}_0$, allowing subsequent inference of clusters for all incoming nodes. While our framework supports spectral and k-means clustering, our primary focus has been on Graph Neural Network (GNN)-based clustering methods. Specifically, we use DMO_N (Tsitsulin et al., 2023; Bianchi, 2022), which maximizes spectral modularity. Modularity (Newman, 2006) measures the divergence between intra-cluster edges and the expected number. These methods use a GNN layer to compute the partition matrix $C = \text{softmax}(\text{MLP}(\tilde{X}, \theta_{\text{MLP}})) \in \mathbb{R}^{N \times K}$, where K is the number of clusters and $\tilde{X}$ is the updated feature embedding generated by one or more message-passing layers. To optimize the C matrix, we minimize the loss function

$$\Delta(C; A) = -\frac{1}{2m} \text{Tr}(C^T B C) + \frac{\sqrt{k}}{n} |\Sigma_i C_i^T|_F - 1$$

which combines spectral modularity maximization with regularization to prevent trivial solutions, where B is the modularity matrix (Tsitsulin et al., 2023). Our static graph $\mathcal{G}_0$ and incoming nodes $\mathcal{E}$ follow Assumption 3.2.

Assumption 3.2. Based on the well-established homophily principle, which forms the basis of most graph coarsening and learning methods. We assume that the generated graphs adhere to the Degree-Corrected Stochastic Block Model (DC-SBM) (Zhao et al., 2012), where intra-class (or intra-community) links are more likely than inter-class links. DC-SBM, an extension of SBM that accounts for degree heterogeneity, making it a more flexible and realistic choice for real-world networks (Details on DC-SBM in Appendix C).

Lemma 3.3. $\mathcal{M}_{\text{clust}}$ Consistency. We adopt the theoretical framework of (Zhao et al., 2012) for a DC-SBM with N nodes and k classes. The edge probability matrix is parameterized as $P_N = \rho_N P$, where $P \in \mathbb{R}^{k \times k}$ is a symmetric matrix containing the between/within community edge probabilities and it is independent of N , $\rho_N = \lambda_N/N$, and λ_N is the average degree of the network. Let $\hat{y}_N = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_N]$ denote the predicted class labels, and let $\hat{C}_N$ be the corresponding $N \times k$ one-hot matrix. Let the true class label matrix is C_N , and μ is any $k \times k$ permutation matrix. Under the adjacency matrix $A^{(N)}$, the global maximum of the objective $\Delta(\cdot; A^{(N)})$ is denoted as $\hat{C}_N^*$. The consistency of class predictions is defined as:

1) Strong Consistency: $P_N \left[\min_{\mu} \|\hat{C}_N^* \mu - C_N\|_F^2 \right] \rightarrow 1$ as $N \rightarrow \infty$.

2) Weak Consistency: $\forall \varepsilon > 0;$
 $P_N \left[\min_{\mu} \frac{1}{N} \|\hat{C}_N^* \mu - C_N\|_F^2 < \varepsilon \right] \rightarrow 1 \text{ as } N \rightarrow \infty.$

where $\|\cdot\|_F$ is the Frobenius norm. Under theorem 3.1 from (Zhao et al., 2012), the $\mathcal{M}_{\text{clust}}$ objective is strongly consistent if $\lambda_N / \log(N) \rightarrow \infty$, and it is weakly consistent when $\lambda_N \rightarrow \infty$.

GraphFLEx focus on learning the structure within each community rather than the structure of the entire dataset at once. Strong consistency ensures perfect community recovery, meaning no inter-community edges exist representing the ideal case. Weak consistency, however, allows for a small fraction (ϵ) of inter-community edges, where ϵ is controlled by ρ_n in $P_n = \rho_n P$, influencing graph sparsity. By Lemma 3.3 and Assumption 3.2, stronger consistency leads to more precise structure learning, whereas weaker consistency permits a limited number of inter-community edges.

3.3 Learning Graph Structure on a Coarse Graph

After training $\mathcal{M}_{\text{clust}}$, we identify communities for incoming nodes, starting with $\tau = 1$. Once assigned, we determine significant communities, i.e., communities with at least one incoming node. We then learn the connections of incoming nodes only to their respective community subgraphs. For large datasets, substantial community sizes may again introduce scalability issues. To mitigate this, we first coarsen the large community graph into a smaller graph and use it to identify potential connections for incoming nodes. This process constitutes the second module of GraphFLEx, denoted as $\mathcal{M}_{\text{coar}}$, which employs LSH-based hashing for graph coarsening. The supernode index for i^{th} node is given as:

$$H_i = \max_{\text{Occurance}} \left\{ \left\lfloor \frac{1}{r} \cdot (\mathcal{W} \cdot X_i + b) \right\rfloor \right\}$$

where r (bin width) controls the coarsened graph size, $\mathcal{W}$ represents random projection matrix, X is the feature matrix, and b is the bias term. For further details, refer to UGC (Kataria et al., 2024). After coarsening the i^{th} community (C_i), $\mathcal{M}_{\text{coar}}(C_i) = \{\mathcal{P}_i, S_i\}$ yields a partition matrix $\mathcal{P}_i \in \mathbb{R}^{\|S_i\| \times \|C_i\|}$ and a set of coarsened supernodes (S_i), as discussed in Section 2. To identify the potential set of nodes for incoming nodes, we define their ideal connections/neighborhood as follows:

Definition 3.4. The neighborhood of a set of nodes $\mathcal{E}_i$ is defined as the union of the top k most similar nodes in C_i for each node $v \in \mathcal{E}_i$, where similarity is measured by the distance function $d(v, u)$. A node $u \in C_i$ is considered part of the neighborhood if its distance $d(v, u)$ is among the k smallest distances for all $u' \in C_i$.

$$\mathcal{N}_k(\mathcal{E}_i) = \bigcup_{v \in \mathcal{E}_i} \{u \in C_i \mid d(v, u) \leq \text{top-}k[d(v, u') : u' \in C_i]\}$$

The neighborhood of incoming nodes $\mathcal{N}_k(\mathcal{E}_i)$ represents the ideal set of nodes where the incoming nodes $\mathcal{E}_i$ are

likely to establish connections when the entire community is provided to a structure learning framework. A robust coarsening framework must reduce the number of nodes within each community C_i while ensuring that the neighborhood of the incoming nodes is preserved.

3.4 Graph Learning only with Potential Nodes

As we now have a coarse representation of the community (Sec. 3.3), we can employ any graph learning algorithms discussed in Section 2.2 to learn a graph between coarsened supernodes S_i and incoming nodes ($V_\tau^i \in V_\tau$). This is the third module of GraphFLEx, i.e., graph learning; we denote it as $\mathcal{M}_{\text{gl}}$. The number of supernodes in S_i is much smaller compared to the original size of the community, i.e., $\|S_i\| \ll \|C_i\|$; scalability is not an issue now. We learn a small graph first using $\mathcal{M}_{\text{gl}}(S_i, X_\tau^i) = \tilde{\mathcal{G}}_\tau^i(V_\tau^c, A_\tau^c)$ where X_τ^i represents features of new incoming nodes belonging to i^{th} community at time τ , $\tilde{\mathcal{G}}_\tau^i(V_\tau^c, A_\tau^c)$ representing the graph between supernodes and incoming nodes. Utilizing the partition matrix $\mathcal{P}_i$ obtained from $\mathcal{M}_{\text{coar}}$, we can precisely determine the set of nodes associated with each supernode. For every new node $V \in V_\tau^i$, we identify the connected supernodes and subsequently select nodes within those supernodes. This subset of nodes is denoted by $\omega_{V_\tau^i}$, the sub-graph associated with $\omega_{V_\tau^i}$ represented by $\mathcal{G}_{\tau-1}^i(\omega_{V_\tau^i})$ then undergoes an additional round of graph learning $\mathcal{M}_{\text{gl}}(\mathcal{G}_{\tau-1}^i(\omega_{V_\tau^i}), X_\tau^i)$, ultimately providing a clear and accurate connection of new nodes V_τ^i with nodes of $\mathcal{G}_{\tau-1}$, ultimately updating it to $\mathcal{G}_\tau$. This multi-step approach, characterized by coarsening, learning on coarsened graphs, and translation to the original graph, ensures scalability.

Theorem 3.5. Neighborhood Preservation. Let $\mathcal{N}_k(\mathcal{E}_i)$ denote the neighborhood of incoming nodes $\mathcal{E}_i$ for the i^{th} community. With partition matrix $\mathcal{P}_i$ and $\mathcal{M}_{\text{gl}}(S_i, X_\tau^i) = \mathcal{G}_\tau^c(V_\tau^c, A_\tau^c)$ we identify the supernodes connected to incoming nodes $\mathcal{E}_i$ and subsequently select nodes within those supernodes; this subset of nodes is denoted by $\omega_{V_\tau^i}$. Formally,

$$\omega_{V_\tau^i} = \bigcup_{v \in \mathcal{E}_i} \left\{ \bigcup_{s \in S_i} \{\pi^{-1}(s) \mid A_\tau^c(v, s) \neq 0\} \right\}$$

Then, with probability $\Pi_{\{c \in \phi\}} p(c)$, it holds that $\mathcal{N}_k(\mathcal{E}_i) \subseteq \omega_{V_\tau^i}$ where

$$p(c) \leq 1 - \frac{2}{\sqrt{2\pi}} \frac{c}{r} \left[1 - e^{-r^2/(2c^2)} \right],$$

and ϕ is a set containing all pairwise distance values ($c = \|v - u\|$) between every node $v \in \mathcal{E}_i$ and the nodes $u \in \omega_{V_\tau^i}$. Here, $\pi^{-1}(s)$ denotes the set of nodes mapped to supernode s , r is the bin-width hyperparameter of $\mathcal{M}_{\text{coar}}$.

Proof. The proof is deferred in Appendix D. $\square$

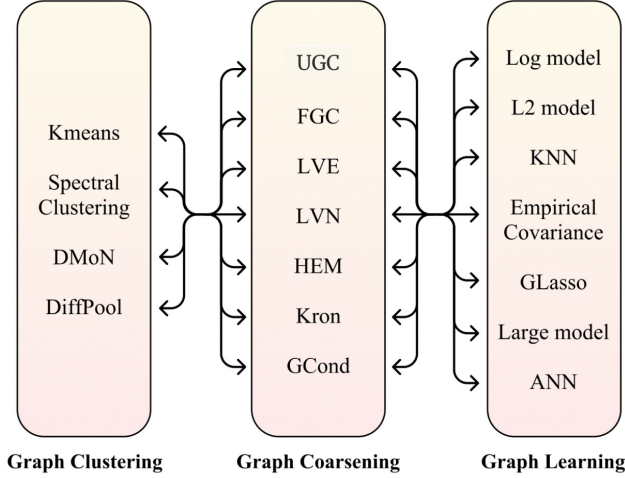


Figure 3. The versatility of GraphFLEx in supporting multiple GSL methods.

Theorem 3.5 establishes that, with a constant probability of success, the neighborhood of incoming nodes $\mathcal{N}_k(\mathcal{E}_i)$ can be effectively recovered using the GraphFLEx multistep approach, which involves coarsening and learning on the coarsened graph, i.e., $\mathcal{N}_k(\mathcal{E}_i) \subseteq \omega_{V_\tau^i}$. The set $\omega_{V_\tau^i}$, estimated by GraphFLEx, identifies potential candidates where incoming nodes are likely to connect. The probability of failure can be reduced by regulating the average degree of connectivity in $\mathcal{M}_{gl}(S_i, X_\tau^i) = \mathcal{G}_\tau^c(V_\tau^c, A_\tau^c)$. While a fully connected network $\mathcal{G}_\tau^c$ ensures all nodes in the community are candidates, it significantly increases computational costs for large communities.

3.5 GraphFLEx: Multiple GSL Frameworks

Each of the three modules of GraphFLEx, as also shown in Figure 3, controls a distinct aspect of the GSL process: clustering influences community detection, coarsening reduces graph complexity via supernodes, and the learning module governs structural inference. Altering any of these modules results in a new GSL method. Currently, we support 48 different graph learning configurations, and this number scales aggressively with the addition of new methods to any module. The number of possible frameworks is given by $\alpha \times \beta \times \gamma$, where α , β , and γ represent the number of clustering, coarsening, and learning methods, respectively.

Module Selection: Since different clustering, coarsening, and learning choices yield distinct GSL variants, we provide module selection guidelines in Appendix L.

3.6 Run Time Analysis

GraphFLEx computational time is always bounded by existing approaches, as it operates on a significantly reduced set of nodes. We evaluate the run-time complexity of Graph-

FLEx in two scenarios: (a) the worst-case scenario, where computationally intensive clustering and coarsening modules are selected, providing an upper bound on time complexity, and (b) the best-case scenario, where the most efficient modules are chosen. Table 1 presents a summary of this analysis for both cases. Due to space limitations, a more comprehensive analysis is provided in Appendix G.

4 Experiments

Tasks and Datasets. To validate GraphFLEx’s utility, we evaluate it across four key dimensions: (i) computational efficiency, (ii) scalability to large graphs, (iii) quality of learned structures, and (iv) adaptability to dynamically growing graphs. To validate the characteristics of GraphFLEx, we conduct extensive experiments on 26 different datasets, including (a) datasets that already have a complete graph structure (allowing comparison between the learned and the original structure), (b) datasets with missing graph structures, (c) synthetic datasets, (d) small datasets for visualizing the graph structure, and (e) large datasets, including datasets with even 2.4M nodes. More details about datasets and system specifications are presented in Table 7 in Appendix A.

4.1 Computational Efficiency.

Existing methods like k -NN and $\log$ -model struggle to learn graph structures even for 20k nodes due to out-of-memory (OOM) or out-of-time (OOT) issues, while $l2$ -model and $large$ -model struggle beyond 50k nodes. Although A -NN and emp -Covar. are faster, GraphFLEx outperforms them on sufficiently large graphs (Table 2). While traditional methods may be efficient for small graphs, GraphFLEx scales significantly better, excelling on large datasets like *Pubmed* and *Syn 5*, where most methods fail. It accelerates structure learning, making A -NN 3× faster and emp -Covar. 2× faster.

4.2 Node Classification Accuracy

Experimental Setup. We now evaluate the prediction performance of GNN models when trained on graph structures learned from three distinct scenarios: 1) *Original Structure*: GNN models trained on the original graph structure, which we refer to as the Base Structure, 2) *GraphFLEx Structure*: GNN models trained on the graph structure learned from GraphFLEx, and 3) *Vanilla Structure*: GNN models trained on the graph structure learned from other existing methods. For each scenario, a unique graph structure is obtained. We trained GNN models on each of these three structure. For more details on GNN model parameters, see Appendix H.

GNN Models. Graph neural networks (GNNs) such as *GCN* (Kipf & Welling, 2016), *GraphSage* (Hamilton et al., 2017), *GIN* (Xu et al., 2018), and *GAT* (Velick-

Table 1. Time complexity analysis of GraphFLEX. Here, N is the number of nodes in the graph, k is the number of nodes in the static subgraph used for clustering ($k \ll N$), and c represents the number of detected communities. k_τ denotes the number of nodes at timestamp τ . Finally, $\alpha = \|S_\tau^i\| + \|\mathcal{E}_\tau^i\|$ is the sum of coarsened and incoming nodes in the relevant community at τ timestamp.

	$\mathcal{M}_{clust}$	$\mathcal{M}_{coar}$	$\mathcal{M}_{gl}$	GraphFLEX
Best (kNN-UGC-ANN)	$\mathcal{O}(k^2)$	$\mathcal{O}\left(\frac{k_\tau}{c}\right)$	$\mathcal{O}(\alpha \log \alpha)$	$\mathcal{O}(k^2 + \frac{k_\tau}{c} + \alpha \log \alpha)$
Worst (SC-FGC-GLasso)	$\mathcal{O}(k^3)$	$\mathcal{O}\left(\left(\frac{k_\tau}{c}\right)^2 \ S_\tau^i\ \right)$	$\mathcal{O}(\alpha^3)$	$\mathcal{O}(k^3 + \left(\frac{k_\tau}{c}\right)^2 \ S_\tau^i\ + \alpha^3)$

Table 2. Computational time(in seconds) for learning graph structures using GraphFLEX (GFlex) with existing methods (Vanilla referred to as Van.). The experimental setup involves treating 50% of the data as static, while the remaining 50% of nodes are treated as incoming nodes arriving in 25 different timestamps. The best times are highlighted by color **Green**. OOM and OOT denote out-of-memory and out-of-time, respectively.

Data	ANN		KNN		log-model		I2-model		emp-Covar.		large-model		Sublime	
	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex
Cora	335	100	8.4	36.1	869	81.6	424	55	8.6	30	2115	18.4	7187	493
Citeseer	1535	454	21.9	75	1113	64.5	977	54.0	14.7	59.2	8319	43.9	8750	670
DBLP	2731	988	OOM	270	77000	919	OOT	1470	359	343	OOT	299	OOM	831
CS	22000	12000	OOM	789	OOT	838	32000	809	813	718	OOT	1469	OOM	1049
PubMed	770	227	OOM	164	OOT	176	OOT	165	488	299	OOT	262	OOM	914
Phy.	61000	21000	OOM	903	OOT	959	OOT	908	2152	1182	OOT	2414	OOM	2731
Syn 3	95	37	OOM	30	58000	346	859	53	88	59	5416	42	6893	780
Syn 4	482	71	OOM	73	OOT	555	OOT	145	2072	1043	OOT	392	OOM	1896

ovic et al., 2017) rely on accurate message passing, dictated by the graph structure, for effective embedding. We use these models to evaluate the above-mentioned learned structures. Table 3 reports node classification performance across all methods. Notably, GraphFLEX outperforms vanilla structures by a significant margin across all datasets, achieving accuracies close to those obtained with the original structure. Figure 8 in Appendix H illustrates *GraphSage* classification results, highlighting GraphFLEX’s superior performance. For the *CS* dataset, GraphFLEX (*large-model*) and GraphFLEX (*empCovar.-model*) even surpass the original structure, demonstrating its ability to preserve key structural properties while denoising edges, leading to improved accuracy.

Graph Transformers. To address limitations of message-passing GNNs (e.g., limited receptive fields and over-smoothing), recent Graph Transformers couple self-attention with structural priors to capture long-range dependencies. We also include node-classification accuracies of Graph Transformer models like GOAT (Kong et al., 2023), Nodeformer (Wu et al., 2022a), and Exoformer (Shirzad et al., 2023) when trained on graph structures learned by GraphFLEX (Table 12 in App. H).

4.3 Scalability of GraphFLEX on Large-Scale Graphs.

To evaluate GraphFLEX’s scalability to large-scale graphs, we consider four datasets with large number of nodes: (a) *Flickr*(89k nodes) (Zeng et al., 2019), (b) *Reddit* (233k nodes) (Zeng et al., 2019), (c) *Ogbn-arxiv* (169k nodes) (Wang et al., 2020), and (d) *Ogbn-products* (2.4M nodes)

(Bhatia et al., 2016). As shown in Table 4, GraphFLEX consistently demonstrates superior scalability across all datasets, outperforming all baseline methods in runtime. In particular, methods such as *log-model*, *I2-model*, and *large-model* fail to run even on *Flickr*, while GraphFLEX successfully scales them on *Flickr*, *Ogbn-arxiv*, and *Reddit*, enabling structure learning where others cannot. For the most computationally demanding dataset, *Ogbn-products*, these methods remain prohibitively expensive even for GraphFLEX. Nonetheless, GraphFLEX efficiently supports scalable structure learning on *Ogbn-products* using the *Covar*, *ANN*, and *KNN* modules. Table 4 also reports node classification accuracy, demonstrating that GraphFLEX maintains performance comparable to the original (base) structure across all datasets. These results confirm that GraphFLEX not only scales effectively, but also preserves the quality of learned structures.

4.4 GraphFLEX for Link Prediction and Graph Classification.

To further validate the generalization, we evaluate GraphFLEX on the link prediction task. Results are in Table 5 which follows the same setting as in Table 3. The structure learned by GraphFLEX demonstrates strong predictive performance, in some cases even outperforming the base structure. This highlights the effectiveness of GraphFLEX in preserving and even enhancing relational information relevant for link prediction.

While our primary focus is on structure learning for node-level tasks, we briefly discuss the applicability of GraphFLEX to graph classification. In such tasks, especially in do-

GraphFLEX: Unsupervised Structure Learning Framework for Large Expanding Graphs

Table 3. Node classification accuracies on different GNN models using GraphFLEX (GFlex) with existing Vanilla (Van.) methods. The experimental setup involves treating 70% of the data as static, while the remaining 30% of nodes are treated as new nodes coming in 25 different timestamps. The best and the second-best accuracies in each row are highlighted by dark and lighter shades of Green, respectively. OOM and OOT denotes out-of-memory and out-of-time respectively.

Data	Model	ANN		KNN		log-model		l2-model		COVAR		large-model		Sublime		Base Struct.
		Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	
DBLP	GAT	34.23	67.37	OOM	69.83	OOT	69.83	OOT	68.98	50.48	68.56	OOT	66.38	OOM	68.32	70.84
	SAGE	34.23	69.58	OOM	70.28	OOT	70.28	OOT	70.68	51.47	70.51	OOT	69.32	OOM	70.28	72.57
	GCN	34.12	69.41	OOM	73.39	OOT	73.39	OOT	73.05	51.50	71.75	OOT	68.55	OOM	69.06	74.43
	GIN	34.01	69.69	OOM	68.19	OOT	68.19	OOT	73.08	52.77	72.03	OOT	71.18	OOM	71.87	73.92
CS	GAT	12.47	60.89	OOM	61.09	OOT	60.95	18.64	61.06	58.96	88.06	OOT	86.22	OOM	64.21	60.75
	SAGE	12.70	78.81	OOM	79.43	OOT	79.06	19.24	78.94	56.97	93.30	OOT	92.79	OOM	78.94	80.33
	GCN	12.59	63.81	OOM	67.94	OOT	69.33	19.21	66.01	58.35	91.07	OOT	84.85	OOM	68.92	67.43
	GIN	13.07	77.62	OOM	78.41	OOT	78.55	19.24	77.61	58.26	92.07	OOT	86.03	OOM	77.61	55.65
Pub.	GAT	49.49	83.71	OOM	84.60	OOT	84.60	OOT	84.04	72.63	83.97	OOT	81.15	OOM	82.15	84.04
	SAGE	50.43	87.27	OOM	87.34	OOT	87.34	OOT	87.42	73.57	86.68	OOT	87.34	OOM	83.45	88.88
	GCN	50.45	82.06	OOM	83.56	OOT	83.56	OOT	83.74	73.14	82.39	OOT	78.03	OOM	70.94	85.54
	GIN	51.82	83.13	OOM	84.31	OOT	84.07	OOT	82.93	73.15	83.51	OOT	82.85	OOM	80.72	86.50
Phy.	GAT	29.18	88.06	OOM	88.47	OOT	88.47	OOT	88.68	58.96	88.06	OOT	86.22	OOM	86.12	88.58
	SAGE	29.57	93.47	OOM	93.47	OOT	93.47	OOT	93.78	56.97	93.60	OOT	92.79	OOM	89.58	94.19
	GCN	27.84	91.27	OOM	91.08	OOT	91.08	OOT	91.78	58.35	91.07	OOT	84.85	OOM	88.46	91.48
	GIN	28.38	92.69	OOM	92.04	OOT	92.04	OOT	92.27	58.26	92.07	OOT	86.03	OOM	87.20	88.89

Table 4. Runtime (sec) and Node Classification Accuracy (%) across large datasets. Each cell shows: **Time / Accuracy**. Van = Vanilla, GFlex = GraphFLEX. OOM = Out of Memory, OOT = Out of Time.

Method	ogbn-arxiv (60.13)		ogbn-products (73.72)		Flickr (44.92)		Reddit (94.15)	
	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex
Covar	OOM —	3.7k — 60.26	OOM —	83.1k — 68.23	2.3k — 44.65	682 — 44.34	OOM —	6.6k — 94.13
ANN	7.8k — 60.14	4.8k — 60.22	OOM —	89.3k — 67.91	2.5k — 44.09	705 — 44.92	12.6k — 94.14	6.1k — 94.18
knn	8.3k — 60.09	6.1k — 60.23	OOM —	91.8k — 68.47	2.7k — 43.95	920 — 44.73	15.6k — 94.14	6.9k — 94.15
l2	OOT —	9.1k — 58.39	OOT —	OOT —	93.3k — 44.90	1.2k — 44.32	OOT —	5.1 — 93.47
log	OOT —	45.6k — 58.72	OOT —	OOT —	OOT —	18.7k — 44.59	OOT —	60.3k — 94.13
large	OOT —	5.6k — 60.20	OOT —	OOT —	OOT —	2.2k — 44.45	OOT —	9.3k — 93.71

Table 5. Link predication accuracy (%) across different datasets. The best and the second-best accuracies in each row are highlighted by dark and lighter shades of Green, respectively.

Data	ANN		KNN		log-model		l2-model		COVAR		large-model		Base Struct.
	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	
DBLP	96.57	96.61	OOM	94.23	OOT	97.59	OOT	97.59	97.22	97.59	OOT	96.24	95.13
Citeseer	80.12	96.32	85.17	96.24	80.48	96.24	80.48	96.48	82.05	96.24	84.50	94.38	90.78
Cora	84.47	95.30	79.23	95.14	90.63	95.45	90.81	95.14	86.05	95.30	90.63	94.67	89.53
Pubmed	94.24	96.91	OOM	97.42	OOT	97.42	OOT	97.37	94.89	94.64	OOT	94.41	94.64
CS	94.21	95.73	OOM	96.02	OOT	93.17	OOT	93.17	93.52	92.31	OOT	95.73	95.00
Physics	95.77	91.34	OOM	94.63	OOT	90.79	OOT	94.63	92.03	90.79	OOT	92.97	93.96

mains like molecule or drug discovery, each data point often corresponds to a small individual subgraph. For these cases, applying clustering and coarsening is typically redundant and may introduce unnecessary computational overhead. Nevertheless, GraphFLEX remains flexible its learning module can be directly used without the clustering or coarsening steps, making it suitable for graph classification as well. This adaptability reinforces GraphFLEX’s utility across a broad range of graph learning tasks.

4.5 Clustering Quality

We measure three metrics to evaluate the resulting clusters or community assignments: a) Normalized Mutual Information (NMI) (Tsitsulin et al., 2023) between the cluster assignments and original labels; b) Conductance (C) (Jerum & Sinclair, 1988) which measures the fraction of total edge volume that points outside the cluster; and c) Modularity (Q) (Newman, 2006) which measures the divergence between the intra-community edges and the expected one. Table 6 illustrates these metrics for single-cell RNA and the MNIST dataset (where the whole structure is missing), and

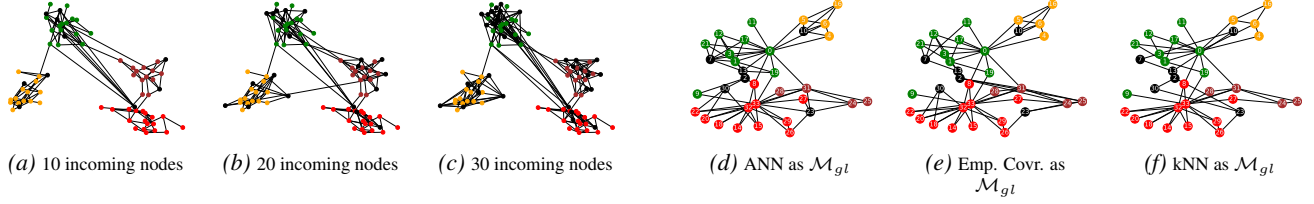


Figure 4. Figures (a), (b), and (c) illustrate the growing structure learned using GraphFLEx for *HE* synthetic dataset. Figures (d), (e), and (f) illustrate the learned structure on Zachary’s karate dataset. New nodes are denoted using black color.

Figure 12 in Appendix K.1 shows the PHATE (Moon et al., 2019) visualization of clusters learned using GraphFLEx’s clustering module $\mathcal{M}_{clust}$. We also train the aforementioned GNN models for the node classification task in order to illustrate the efficacy of the learned structures; the accuracy values presented in Table 6, clearly highlight the significance of the learned structures, as reflected by the high accuracy values. For a comprehensive ablation study, refer to Appendix K.

Table 6. Clustering (NMI, $\mathcal{C}$, $\mathcal{Q}$) and node classification accuracy using GCN, GraphSAGE, GIN, and GAT.

Data	NMI	$\mathcal{C}$	$\mathcal{Q}$	GCN	SAGE	GIN	GAT
Bar. M.	0.716	0.057	0.741	91.2	96.2	95.1	94.9
Seger.	0.678	0.102	0.694	91.0	93.9	94.2	92.3
Mura.	0.843	0.046	0.706	96.9	97.4	97.5	96.4
Bar. H.	0.674	0.078	0.749	95.3	96.4	97.2	95.8
Xin	0.741	0.045	0.544	98.6	99.3	98.9	99.8
MNIST	0.677	0.082	0.712	92.9	94.5	94.9	82.6

4.6 Structure Visualization

We evaluate the structures generated by GraphFLEx through visualizations on four small datasets: (i) MNIST (LeCun et al., 2010), consisting of handwritten digit images, where Figure 5(a) shows that images of the same digit are mostly connected; (ii) Pre-trained GloVe embeddings (Pennington et al., 2014) of English words, with Figure 5(b) revealing that frequently used words are closely connected; (iii) A synthetic *HE* dataset (see Appendix A), demonstrating GraphFLEx’s ability to handle expanding networks without requiring full relearning. Figure 4(a-c) shows the graph structure evolving as 30 new nodes are added over three timestamps; and (iv) Zachary’s karate club network (Zachary, 1977), which highlights GraphFLEx’s multi-framework capability. Figure 4(d-f) shows three distinct graph structures after altering the learning module.

5 Conclusion

Large or expanding graphs challenge the best of graph learning approaches. GraphFLEx, introduced in this paper, seamlessly adds new nodes into an existing graph structure.

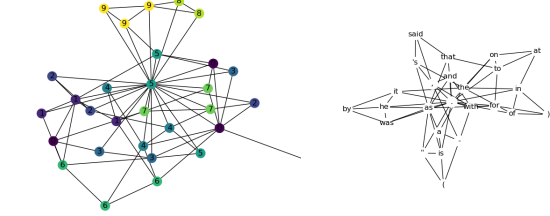


Figure 5. Effectiveness of GraphFLEx in learning structure between similar MNIST digits and GloVe embeddings.

It offers diverse methods for acquiring the graph’s structure. GraphFLEx consists of three key modules: Clustering, Coarsening, and Learning which empowers GraphFLEx to serve as a comprehensive framework applicable individually for clustering, coarsening, and learning tasks. Empirically, GraphFLEx outperforms state-of-the-art baselines, achieving up to 3× speedup while preserving structural quality. The performance across multiple real and synthetic datasets affirms the utility and efficacy of GraphFLEx for graph structure learning.

6 Acknowledgement

This work is supported by Yardi-ScAI, IIT Delhi research fund.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Allen, J. D., Xie, Y., Chen, M., Girard, L., and Xiao, G. Comparing statistical methods for constructing large scale gene networks. *PloS one*, 7(1):e29348, 2012.
- Banerjee, O., El Ghaoui, L., and d’Aspremont, A. Model selection through sparse maximum likelihood estimation for multivariate gaussian or binary data. *The Journal of Machine Learning Research*, 9:485–516, 2008.

- Belkin, M., Niyogi, P., and Sindhvani, V. Manifold regularization: A geometric framework for learning from labeled and unlabeled examples. *Journal of machine learning research*, 7(11), 2006.
- Bhatia, K., Dahiya, K., Jain, H., Kar, P., Mittal, A., Prabhu, Y., and Varma, M. The extreme classification repository: Multi-label datasets and code, 2016. URL <http://manikvarma.org/downloads/XC/XMLRepository.html>.
- Bianchi, F. M. Simplifying clustering with graph neural networks. *arXiv preprint arXiv:2207.08779*, 2022.
- Bruna, J., Zaremba, W., Szlam, A., and LeCun, Y. Spectral networks and deep locally connected networks on graphs. arxiv. *arXiv preprint arXiv:1312.6203*, 2014.
- Chen, Y. and Wu, L. Graph neural networks: Graph structure learning. *Graph Neural Networks: Foundations, Frontiers, and Applications*, pp. 297–321, 2022.
- Datar, M., Immorlica, N., Indyk, P., and Mirrokni, V. S. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the twentieth annual symposium on Computational geometry*, pp. 253–262, 2004.
- Defferrard, M., Martin, L., Pena, R., and Perraudin, N. Pygsp: Graph signal processing in python. URL <https://github.com/epfl-lts2/pygsp/>.
- Defferrard, M., Bresson, X., and Vandergheynst, P. Convolutional neural networks on graphs with fast localized spectral filtering. *Advances in neural information processing systems*, 29, 2016.
- Dempster, A. P. Covariance selection. *Biometrics*, pp. 157–175, 1972.
- Dong, W., Moses, C., and Li, K. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th international conference on World wide web*, pp. 577–586, 2011.
- Dong, X., Thanou, D., Frossard, P., and Vandergheynst, P. Learning laplacian matrix in smooth graph signal representations. *IEEE Transactions on Signal Processing*, 64(23):6160–6173, 2016.
- Fatemi, B., El Asri, L., and Kazemi, S. M. Slaps: Self-supervision improves structure learning for graph neural networks. *Advances in Neural Information Processing Systems*, 34:22667–22681, 2021.
- Fortunato, S. Community detection in graphs. *Physics reports*, 486(3-5):75–174, 2010.
- Fout, A., Byrd, J., Shariat, B., and Ben-Hur, A. Protein interface prediction using graph convolutional networks. *Advances in neural information processing systems*, 30, 2017.
- Friedman, J., Hastie, T., and Tibshirani, R. Sparse inverse covariance estimation with the graphical lasso. *Biostatistics*, 9(3):432–441, 2008.
- Fu, X., Zhang, J., Meng, Z., and King, I. Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of The Web Conference 2020*, pp. 2331–2341, 2020.
- Gu, J., Zhao, H., Lin, Z., Li, S., Cai, J., and Ling, M. Scene graph generation with external knowledge and image reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 1969–1978, 2019.
- Hamilton, W., Ying, Z., and Leskovec, J. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- Hashemi, M., Gong, S., Ni, J., Fan, W., Prakash, B. A., and Jin, W. A comprehensive survey on graph reduction: Sparsification, coarsening, and condensation. *arXiv preprint arXiv:2402.03358*, 2024.
- Hsieh, C.-J., Dhillon, I., Ravikumar, P., and Sustik, M. Sparse inverse covariance matrix estimation using quadratic approximation. *Advances in neural information processing systems*, 24, 2011.
- Hu, C., Cheng, L., Sepulcre, J., El Fakhri, G., Lu, Y. M., and Li, Q. A graph theoretical regression model for brain connectivity learning of alzheimer’s disease. In *2013 IEEE 10th International Symposium on Biomedical Imaging*, pp. 616–619. IEEE, 2013.
- Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., and Leskovec, J. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33:22118–22133, 2020.
- Jerrum, M. and Sinclair, A. Conductance and the rapid mixing property for markov chains: the approximation of permanent resolved. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, pp. 235–244, 1988.
- Jin, W., Ma, Y., Liu, X., Tang, X., Wang, S., and Tang, J. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 66–74, 2020.

- Jin, W., Zhao, L., Zhang, S., Liu, Y., Tang, J., and Shah, N. Graph condensation for graph neural networks. *arXiv preprint arXiv:2110.07580*, 2021.
- Kalofolias, V. How to learn a graph from smooth signals. In *Artificial intelligence and statistics*, pp. 920–929. PMLR, 2016.
- Kamvar, S. D., Klein, D., and Manning, C. D. Spectral learning. In *IJCAI*, volume 3, pp. 561–566, 2003.
- Kataria, M., Khandelwal, A., Das, R., Kumar, S., and Jayadeva, J. Linear complexity framework for feature-aware graph coarsening via hashing. In *NeurIPS 2023 Workshop: New Frontiers in Graph Learning*, 2023. URL <https://openreview.net/forum?id=HKdsrm5nCW>.
- Kataria, M., Kumar, S., and Jayadeva, J. UGC: Universal graph coarsening. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=nN6NSdlQds>.
- Kim, S., Yun, S., and Kang, J. Dygrain: An incremental learning framework for dynamic graphs. In *IJCAI*, pp. 3157–3163, 2022.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- Koller, D. and Friedman, N. *Probabilistic graphical models: principles and techniques*. MIT press, 2009.
- Kong, K., Chen, J., Kirchenbauer, J., Ni, R., Bruss, C. B., and Goldstein, T. Goat: A global transformer on large-scale graphs. In *International Conference on Machine Learning*, pp. 17375–17390. PMLR, 2023.
- Kumar, M., Sharma, A., and Kumar, S. A unified framework for optimization-based graph coarsening. *Journal of Machine Learning Research*, 24(118):1–50, 2023. URL <http://jmlr.org/papers/v24/22-1085.html>.
- Lao, D., Yang, X., Wu, Q., and Yan, J. Variational inference for training graph neural networks in low-data regime through joint structure-label estimation. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, pp. 824–834, 2022.
- LeCun, Y., Cortes, C., and Burges, C. Mnist handwritten digit database. *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, 2, 2010.
- Liu, N., Wang, X., Wu, L., Chen, Y., Guo, X., and Shi, C. Compact graph structure learning via mutual information compression. In *Proceedings of the ACM web conference 2022*, pp. 1601–1610, 2022a.
- Liu, Y., Zheng, Y., Zhang, D., Chen, H., Peng, H., and Pan, S. Towards unsupervised deep graph structure learning. In *Proceedings of the ACM Web Conference 2022*, pp. 1392–1403, 2022b.
- Loukas, A. Graph reduction with spectral and cut guarantees. *J. Mach. Learn. Res.*, 20(116):1–42, 2019.
- Lü, L. and Zhou, T. Link prediction in complex networks: A survey. *Physica A: statistical mechanics and its applications*, 390(6):1150–1170, 2011.
- MacQueen, J. et al. Some methods for classification and analysis of multivariate observations. In *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, volume 1, pp. 281–297. Oakland, CA, USA, 1967.
- Malik, N., Gupta, R., and Kumar, S. Hyperdefender: A robust framework for hyperbolic gnns. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(18):19396–19404, Apr. 2025. doi: 10.1609/aaai.v39i18.34135. URL <https://ojs.aaai.org/index.php/AAAI/article/view/34135>.
- Moon, K. R., Van Dijk, D., Wang, Z., Gigante, S., Burkhardt, D. B., Chen, W. S., Yim, K., Elzen, A. v. d., Hirn, M. J., Coifman, R. R., et al. Visualizing structure and transitions in high-dimensional biological data. *Nature biotechnology*, 37(12):1482–1492, 2019.
- Muja, M. and Lowe, D. G. Scalable nearest neighbor algorithms for high dimensional data. *IEEE transactions on pattern analysis and machine intelligence*, 36(11):2227–2240, 2014.
- Newman, M. E. Modularity and community structure in networks. *Proceedings of the national academy of sciences*, 103(23):8577–8582, 2006.
- Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., and Wermter, S. Continual lifelong learning with neural networks: A review. *Neural networks*, 113:54–71, 2019.
- Pennington, J., Socher, R., and Manning, C. D. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 1532–1543, 2014.
- Shchur, O., Mumme, M., Bojchevski, A., and Günnemann, S. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868*, 2018.
- Shirzad, H., Velingker, A., Venkatachalam, B., Sutherland, D. J., and Sinop, A. K. Expformer: Sparse transformers for graphs. In *International Conference on Machine Learning*, pp. 31613–31632. PMLR, 2023.

- Tsitsulin, A., Palowitch, J., Perozzi, B., and Müller, E. Graph clustering with graph neural networks. *Journal of Machine Learning Research*, 24(127):1–21, 2023.
- Van de Ven, G. M. and Tolias, A. S. Three scenarios for continual learning. *arXiv preprint arXiv:1904.07734*, 2019.
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., et al. Graph attention networks. *stat*, 1050(20):10–48550, 2017.
- Vogelstein, J. T., Roncal, W. G., Vogelstein, R. J., and Priebe, C. E. Graph classification using signal-subgraphs: Applications in statistical connectomics. *IEEE transactions on pattern analysis and machine intelligence*, 35(7):1539–1551, 2012.
- Wagstaff, K., Cardie, C., Rogers, S., Schrödl, S., et al. Constrained k-means clustering with background knowledge. In *Icml*, volume 1, pp. 577–584, 2001.
- Wang, F. and Zhang, C. Label propagation through linear neighborhoods. In *Proceedings of the 23rd international conference on Machine learning*, pp. 985–992, 2006.
- Wang, K., Shen, Z., Huang, C., Wu, C.-H., Dong, Y., and Kanakia, A. Microsoft academic graph: When experts are not enough. *Quantitative Science Studies*, 1(1):396–413, 2020.
- Wang, R., Mou, S., Wang, X., Xiao, W., Ju, Q., Shi, C., and Xie, X. Graph structure estimation neural networks. In *Proceedings of the web conference 2021*, pp. 342–353, 2021.
- Watts, D. J. and Strogatz, S. H. Collective dynamics of ‘small-world’ networks. *nature*, 393(6684):440–442, 1998.
- Wu, Q., Zhao, W., Li, Z., Wipf, D. P., and Yan, J. Nodeformer: A scalable graph structure learning transformer for node classification. *Advances in Neural Information Processing Systems*, 35:27387–27401, 2022a.
- Wu, Q., Zhao, W., Li, Z., Wipf, D. P., and Yan, J. Nodeformer: A scalable graph structure learning transformer for node classification. *Advances in Neural Information Processing Systems*, 35:27387–27401, 2022b.
- Wu, T., Liu, Q., Cao, Y., Huang, Y., Wu, X.-M., and Ding, J. Continual graph convolutional network for text classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 13754–13762, 2023.
- Wu, Y., Lian, D., Xu, Y., Wu, L., and Chen, E. Graph convolutional networks with markov random field reasoning for social spammer detection. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 1054–1061, 2020.
- Xiang, L., Yuan, Q., Zhao, S., Chen, L., Zhang, X., Yang, Q., and Sun, J. Temporal recommendation on graphs via long-and short-term preference fusion. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 723–732, 2010.
- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*, 2018.
- Yang, F., Wang, W., Wang, F., Fang, Y., Tang, D., Huang, J., Lu, H., and Yao, J. scbert as a large-scale pretrained deep language model for cell type annotation of single-cell rna-seq data. *Nature Machine Intelligence*, 4(10):852–866, 2022.
- Yang, Z., Cohen, W., and Salakhudinov, R. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, pp. 40–48. PMLR, 2016.
- You, J., Du, T., and Leskovec, J. Roland: graph learning framework for dynamic graphs. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, pp. 2358–2366, 2022.
- Zachary, W. W. An information flow model for conflict and fission in small groups. *Journal of anthropological research*, 33(4):452–473, 1977.
- Zeng, H., Zhou, H., Srivastava, A., Kannan, R., and Prasanna, V. Graphsaint: Graph sampling based inductive learning method. *arXiv preprint arXiv:1907.04931*, 2019.
- Zhang, X., Song, D., and Tao, D. Cglb: Benchmark tasks for continual graph learning. *Advances in Neural Information Processing Systems*, 35:13006–13021, 2022.
- Zhao, Y., Levina, E., and Zhu, J. Consistency of community detection in networks under degree-corrected stochastic block models. 2012.
- Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., and Sun, M. Graph neural networks: A review of methods and applications. *AI open*, 1:57–81, 2020.
- Zhu, X., Ghahramani, Z., and Lafferty, J. D. Semi-supervised learning using gaussian fields and harmonic functions. In *Proceedings of the 20th International conference on Machine learning (ICML-03)*, pp. 912–919, 2003.
- Zhu, Y., Xu, W., Zhang, J., Du, Y., Zhang, J., Liu, Q., Yang, C., and Wu, S. A survey on graph structure learning: Progress and opportunities. *arXiv preprint arXiv:2103.03036*, 2021.

Appendix

A Datasets

Datasets used in our experiments vary in size, with nodes ranging from 1k to 60k. Table 7 lists all the datasets we used in our work. We evaluate our proposed framework *GraphFlex* on real-world datasets *Cora*, *Citeseer*, *Pubmed* (Yang et al., 2016), *CS*, *Physics* (Shchur et al., 2018), *DBLP* (Fu et al., 2020), all of which include graph structures. These datasets allow us to compare the learned structures with the originals. Additionally, we utilize single-cell RNA pancreas datasets (Yang et al., 2022), including Baron, Muraro, Segerstolpe, and Xin, where the graph structure is missing. The Baron dataset was downloaded from the Gene Expression Omnibus (GEO) (accession no. GSE84133). The Muraro dataset was downloaded from GEO (accession no. GSE85241). The Segerstolpe dataset was accessed from ArrayExpress (accession no. E-MTAB-5061). The Xin dataset was downloaded from GEO (accession no. GSE81608). We simulate the expanding graph scenario by splitting the original dataset across different $\mathcal{T}$ timestamps. We assumed 50% of the nodes were static, with the remaining nodes arriving as incoming nodes at different timestamps.

Synthetic datasets: Different data generation techniques validate that our results are generalized to different settings.

Table 7. Summary of the datasets.

Category	Data	Nodes	Edges	Feat.	Class	Type
Original Structure Known	Cora	2,708	5,429	1,433	7	Citation network
	Citeseer	3,327	9,104	3,703	6	Citation network
	DBLP	17,716	52.8k	1,639	4	Research paper
	CS	18,333	163.7k	6,805	15	Co-authorship network
	PubMed	19,717	44.3k	500	3	Citation network
	Physics	34,493	247.9k	8,415	5	Co-authorship network
Original Structure Not Known	Xin	1,449	NA	33,889	4	Human Pancreas
	Baron Mouse	1,886	NA	14,861	13	Mouse Pancreas
	Muraro	2,122	NA	18,915	9	Human Pancreas
	Segerstolpe	2,133	NA	22,757	13	Human Pancreas
	Baron Human	8,569	NA	17,499	14	Human Pancreas
Synthetic	Syn 1	2,000	8,800	150	4	SW
	Syn 2	5,000	22k	150	4	SW
	Syn 3	10,000	44k	150	7	SW
	Syn 4	50,000	220k	150	7	SW
	Syn 5	400	1,520	100	4	PyG-G
	Syn 6	2,500	9,800	100	4	PyG-S
	Syn 7	1,000	9,990	150	4	HE
	Syn 8	2,000	40k	150	4	HE
Visulization Datasets	MNIST	60,000	NA	784	10	Images
	Zachary’s karate	34	156	34	4	Karate club network
	Glove	2,000	NA	50	NA	GloVe embeddings
Large dataset	Flickr	89,250	899,756	500	7	-
	Reddit	232,965	11.60M	602	41	-
	Ogbn-arxiv	169,343	1.16M	128	40	-
	Ogbn-products	2,449,029	61.85M	100	47	-

Please refer to Table 7 for more details about the number of nodes, edges, features, and classes, *Syn* denotes the type of synthetic datasets. Figure 6 shows graphs generated using different methods. We have employed three different ways to generate synthetic datasets which are mentioned below:

- **PyGSP(PyGsp):** We used synthetic graphs created by PyGSP (Defferrard et al.) library. PyG-G and PyG-S denotes grid and sensor graphs from PyGSP.
- **Watts–Strogatz’s small world(SW):** (Watts & Strogatz, 1998) proposed a generation model that produces graphs with small-world properties, including short average path lengths and high clustering.
- **Heterophily(HE):** We propose a method for creating synthetic datasets to explore graph behavior across a heterophily

spectrum by manipulating heterophilic factor α , and classes. α is determined by dividing the number of edges connecting nodes from different classes by the total number of edges in the graph.

Visualization Datasets: To evaluate, the learned graph structure, we have also included three datasets: (i) MNIST (LeCun et al., 2010), consisting of handwritten digit images; (ii) Pre-trained GloVe embeddings (Pennington et al., 2014) of English words; and (iii) Zachary’s karate club network (Zachary, 1977).

Large Datasets: To comprehensively evaluate GraphFLEx’s scalability to large-scale graphs, we consider four datasets with a high number of nodes: (a) Flickr(89k nodes) (Zeng et al., 2019), (b) Reddit (233k nodes) (Zeng et al., 2019), (c) Ogbn-arxiv (169k nodes) (Wang et al., 2020), and (d) Ogbn-products (2.4M nodes) (Bhatia et al., 2016).

System Specifications: All the experiments conducted for this work were performed on an Intel Xeon W-295 CPU with 64GB of RAM desktop using the Python environment.

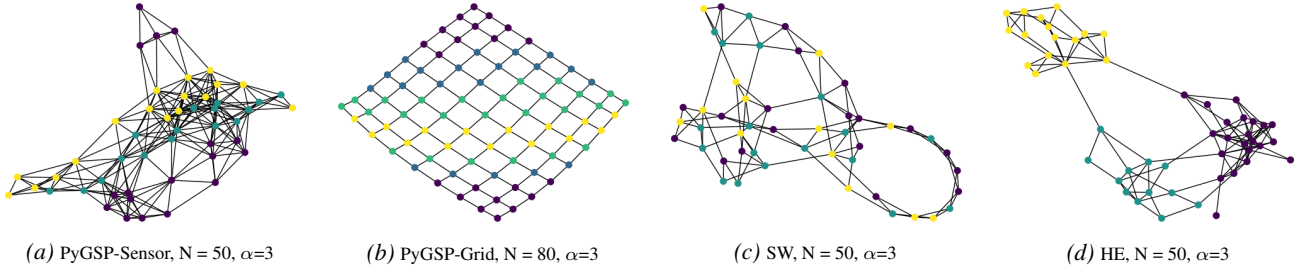


Figure 6. This figure illustrates different types of synthetic graphs generated using i)PyGSP, ii) Watts–Strogatz’s small world(SW), and iii) Heterophily(HE). N denotes the number of nodes, while α denotes the number of classes.

B Algorithm

In this section we include the pseudocode of GraphFLEx; see Algorithm 1.

C Degree-Corrected Stochastic Block Model(DC-SBM)

The DC-SBM is one of the most commonly used models for networks with communities and postulates that, given node labels $\mathbf{c} = c_1, \dots, c_n$, the edge variables A'_{ij} s are generated via the formula

$$E[A_{ij}] = \theta_i \theta_j P_{c_i} P_{c_j}$$

, where θ_i is a ”degree parameter” associated with node i , reflecting its individual propensity to form ties, and P is a $K \times K$ symmetric matrix containing the between/withincommunity edge probabilities and $P_{c_i} P_{c_j}$ denotes the edge probabilities between community c_i and c_j .

For DC-SBM model (Zhao et al., 2012) assumed P_n on n nodes with k classes, each node v_i is given a label/degree pair (c_i, θ_i) , drawn from a discrete joint distribution $\Pi_{K \times m}$ which is fixed and does not depend on n . This implies that each θ_i is one of a fixed set of values $0 \leq x_1 \leq \dots \leq x_m$. To facilitate analysis of asymptotic graph sparsity, we parameterize the edge probability matrix P as $P_n = \rho_n P$ where P is independent of n , and $\rho_n = \lambda_n/n$ where λ_n is the average degree of the network.

D Neighbourhood Preservation

Theorem D.1. Neighborhood Preservation. Let $\mathcal{N}_k(\mathcal{E}_i)$ denote the neighborhood of incoming nodes $\mathcal{E}_i$ for the i^{th} community. With partition matrix $\mathcal{P}_i$ and $\mathcal{M}_{gl}(S_i, X_\tau^i) = \mathcal{G}_\tau^c(V_\tau^c, A_\tau^c)$ we identify the supernodes connected to incoming nodes $\mathcal{E}_i$ and subsequently select nodes within those supernodes; this subset of nodes is denoted by $\omega_{V_\tau^i}$. Formally,

$$\omega_{V_\tau^i} = \bigcup_{v \in \mathcal{E}_i} \left\{ \bigcup_{s \in S_i} \{\pi^{-1}(s) | A_\tau^c(v, s) \neq 0\} \right\}$$

Algorithm 1 GraphFlex: A Unified Structure Learning framework for expanding and Large Scale Graphs

Input: Graph $G_0(X_0, A_0)$, expanding nodes set $\mathcal{E}_1^T = \{\mathcal{E}_\tau(\mathcal{V}_\tau, \mathcal{X}_\tau)\}_{\tau=1}^T$

Parameter: GClust, GCoar, GL $\leftarrow$ Clustering, Coarsening and Learning Module

Output: Graph $G_T(X_T, A_T)$

```

1: Train clustering module  $train(\mathcal{M}_{clust}, \text{GClust}, G_0)$ 
2: for each  $E_t(V_t, X_t)$  in  $\mathcal{E}_1^T$  do
3:    $C_t = infer(\mathcal{M}_{clust}, X_t)$ ,  $C_t \in \mathbb{R}^{N_t}$  denotes the communities of  $N_t$  nodes at time  $t$ .
4:    $I_t = unique(C_t)$ .
5:   for each  $I_t^i$  in  $I_t$  do
6:      $G_{t-1}^i = \text{subgraph}(G_{t-1}, I_t^i)$ 
7:      $\{S_{t-1}^i, P_{t-1}^i\} = \mathcal{M}_{coar}(G_{t-1}^i)$ ,  $S_{t-1}^i \in \mathbb{R}^{k \times d}$  are features of  $k$  supernodes,  $P_{t-1}^i \in \mathbb{R}^{k \times N_t^i}$  is the partition matrix.
8:      $Gc_{t-1}^i(S_{t-1}^i, A_{t-1}^i) = \mathcal{M}_{gl}(S_{t-1}^i, X_t^i)$ ,  $Gc_{t-1}^i$  is the learned graph on super-nodes  $S_{t-1}^i$  and new node  $X_t^i$ .
9:      $\omega_t^i \leftarrow []$ 
10:    for  $x \in X_t^i$  do
11:       $\omega_t^i.append(x)$ 
12:       $n_p = \{n \mid A_{t-1}^i[n] > 0\}$ 
13:       $\omega_t^i.append(n_p)$ 
14:    end for
15:     $G_{t-1} = update(G_{t-1}, \mathcal{M}_{gl}(\omega_t^i))$ 
16:  end for
17:   $G_t = G_{t-1}$ 
18: end for
19: return  $G_T(X_T, A_T) = 0$ 
    
```

Then, with probability $\Pi_{\{c \in \phi\}} p(c)$, it holds that $\mathcal{N}_k(\mathcal{E}_i) \subseteq \omega_{V_\tau^i}$ where

$$p(c) \leq 1 - \frac{2}{\sqrt{2\pi}} \frac{c}{r} \left[1 - e^{-r^2/(2c^2)} \right],$$

and ϕ is a set containing all pairwise distance values ($c = \|v - u\|$) between every node $v \in \mathcal{E}_i$ and the nodes $u \in \omega_{V_\tau^i}$. Here, $\pi^{-1}(s)$ denotes the set of nodes mapped to supernode s , r is the bin-width hyperparameter of $\mathcal{M}_{coar}$.

Proof: The probability that LSH random projection (Kataria et al., 2024; Datar et al., 2004) preserves the distance between two nodes v and u i.e., $d(u, v) = c$, is given by:

$$p(c) = \int_0^r \frac{1}{c} f_2\left(\frac{t}{c}\right) \left(1 - \frac{t}{r}\right) dt,$$

where $f_2(x) = \frac{2}{\sqrt{2\pi}} e^{-x^2/2}$ represents the Gaussian kernel when the projection matrix is randomly sampled from p -stable($p = 2$) distribution (Datar et al., 2004).

The probability $p(c)$ can be decomposed into two terms:

$$p(c) = S_1(c) - S_2(c),$$

$S_1(c)$ and $S_2(c)$ are defined as follows:

$$S_1(c) = \frac{2}{\sqrt{2\pi}} \int_0^r e^{-(t/c)^2/2} dt \leq 1,$$

$$S_2(c) = \frac{2}{\sqrt{2\pi}} \int_0^r e^{-(t/c)^2/2} \frac{t}{r} dt.$$

$$S_2(c) = \frac{2}{\sqrt{2\pi}} \cdot \frac{c}{r} \int_0^r e^{-(t/c)^2/2} \frac{t}{c^2} dt$$

Expanding $S_2(c)$:

$$S_2(c) = \frac{2}{\sqrt{2\pi}} \cdot \frac{c}{r} \int_0^{r^2/(2c^2)} e^{-y} dy$$

$$S_2(c) = \frac{2}{\sqrt{2\pi}} \cdot \frac{c}{r} \left[1 - e^{-r^2/(2c^2)} \right]$$

Thus, the probability $p(c)$ can be bounded as:

$$p(c) \leq 1 - \frac{2}{\sqrt{2\pi}} \frac{c}{r} \left[1 - e^{-r^2/(2c^2)} \right].$$

Now, let ϕ be the set of all pairwise distances $d(u, v)$, where $v \in \mathcal{E}_i$ and node $\omega_{V_\tau^i}$. The probability that all nodes in $\mathcal{N}_k(\mathcal{E}_i)$ are preserved within $\omega_{V_\tau^i}$, requires that all distances $c \in \phi$ are also preserved. The probability is then given by:

$$\prod_{c \in \phi} p(c).$$

$$\prod_{c \in \phi} p(c) \leq \prod_{c \in \phi} \left(1 - \frac{2}{\sqrt{2\pi}} \frac{c}{r} \left[1 - e^{-r^2/(2c^2)} \right] \right).$$

E Continual Learning and Dynamic Graph Learning

In this subsection, we highlight the key distinctions between Graph Structure Learning (GSL) and related fields to justify our specific selection of related works in Section 2.2. GSL is often confused with topics such as Continual Learning (CL) and Dynamic Graph Learning (DGL).

CL (Van de Ven & Tolias, 2019; Zhang et al., 2022; Parisi et al., 2019) addresses the issue of catastrophic forgetting, where a model’s performance on previously learned tasks degrades significantly after training on new tasks. In CL, the model has access only to the current task’s data and cannot utilize data from prior tasks. Conversely, DGL (Kim et al., 2022; Wu et al., 2023; You et al., 2022) focuses on capturing the evolving structure of graphs and maintaining updated graph representations, with access to all prior information.

While both CL and DGL aim to *enhance model adaptability* to dynamic data, GSL is primarily concerned with generating *high-quality graph structures* that can be leveraged for downstream tasks such as node classification (Kipf & Welling, 2016), link prediction (Lü & Zhou, 2011), and graph classification (Vogelstein et al., 2012). Moreover, in CL and DGL, different tasks typically involve distinct data distributions, whereas GSL assumes a consistent data distribution throughout.

We agree that supervised GSL methods tightly couple structure refinement with GNN training. In such settings, one could replace static GNNs with dynamic or continual GNNs to better handle evolving graphs. We recognize that extending GraphFLEx-style structure learning with dynamic GNNs for s-GSL is a promising direction for future work.

F Related Work

F.1 uGSL and sGSL

GraphFLEx is designed for unsupervised graph structure learning (u-GSL), this places GraphFLEx fundamentally apart from supervised GSL methods such as *Pro-GNN* (Jin et al., 2020), *SLAPS* (Fatemi et al., 2021), *GEN* (Wang et al., 2021), *CoGSL* (Liu et al., 2022a), *NodeFormer* (Wu et al., 2022b), etc., which require labels, assume an initial graph, and jointly optimize structure and task performance. Table 8 contains key distinctions between u-GSL and s-GSL.

Scalability of supervised GSL (s-GSL) methods: For completeness, we ran representative supervised GSL methods (e.g., *Pro-GNN* (Jin et al., 2020), *SLAPS* (Fatemi et al., 2021), *CoGSL* (Liu et al., 2022a)) under the same compute budget (see Table 9). These methods are *not directly comparable* to our u-GSL setting because they use supervision to optimize structure and typically assume an observed base graph. We therefore report only computational outcomes (runtime / OOM) to provide context on scalability. We provided these methods with the *observed base graph* (as required by their formulations), since s-GSL approaches typically refine an existing adjacency rather than infer structure from scratch.

Table 10 presents the formulations and associated time complexities of various unsupervised Graph Structure Learning methods.

Table 8. Key distinctions between unsupervised GSL (u-GSL) and supervised GSL (s-GSL).

Aspect	Unsupervised GSL (u-GSL)	Supervised GSL (s-GSL)
Use of Labels	No labels required	Requires labels
Initial Graph	May be fully missing	Typically available (noisy / refinement)
Objective	Learn structure only (adjacency A or Laplacian L)	Improve downstream task (e.g., classification)
Dependence on GNN	Independent of GNN training	Joint optimization with GNN
Examples	kNN, A-NN, GLasso, SUBLIME, $\ell_2/\log/\text{large-model}$	ProGNN, SLAPS, NodeFormer, GEN, CoGSL

Table 9. Runtime (seconds) of representative s-GSL methods under a fixed compute budget. These results are reported for scalability context only; s-GSL methods rely on supervision and are not directly comparable to u-GSL.

Dataset	Pro-GNN	SLAPS	CoGSL
CS	< 40k	< 40k	< 40k
Citeseer	3421	2490	3734
Cora	2981	2138	2642
DBLP	< 40k	14579	< 40k
Physics	< 40k	< 40k	< 40k

G Run Time Analysis

In the context of clustering module, $k - NN$ is the fastest algorithm, while Spectral Clustering is the slowest. Suppose we aim to learn the structure of a graph with N nodes. The clustering module, however, is only applied to a randomly sampled, smaller, static subgraph with k nodes, where $k \ll N$. In the worst-case scenario, spectral clustering requires $\mathcal{O}(k^3)$ time, whereas in the best case, $k - NN$ requires $\mathcal{O}(k^2)$ time. For coarsening module, LSH-based coarsening framework (Kataria et al., 2023), has the best time complexity of $\mathcal{O}(\frac{k_\tau}{c})$ while FGC denotes the worst case with a time-complexity of $\mathcal{O}((\frac{k_\tau}{c})^2 \|S_\tau^i\|)$ where c is the number of communities detected by clustering module $\mathcal{M}_{\text{clust}}$, $\|S_\tau^i\|$ is the number of coarsened node in the relevant community at τ timestamp and k_τ denotes number of nodes at τ timestamp. For learning module, $A - NN$ is the most efficient algorithm with time complexity as $\mathcal{O}(N \log N)$, while $GLasso$ has the worst computational cost of $\mathcal{O}(N^3)$. So, the effective time complexity of GraphFLEx is upper bounded by $\mathcal{O}(k^3 + (\frac{k_\tau}{c})^2 \|S_\tau^i\| + \alpha^3)$ and lower bounded by $\mathcal{O}(k^2 + \frac{k_\tau}{c} + \alpha \log \alpha)$ where $\alpha = \|S_\tau^i\| + \|\mathcal{E}_\tau^i\|$. GraphFLEx’s efficiency in term of computational time is evident in Figure 1 and further quantified in Table 2.

Out of the three modules of GraphFLEx first module ($\mathcal{M}_{\text{clust}}$) is trained once, and hence its run time is always bounded; computational time for second module ($\mathcal{M}_{\text{coar}}$) can also be controlled because some of the methods either needs training once (Jin et al., 2021) or have linear time complexity (Kataria et al., 2023). Consequently, both the clustering and coarsening modules contribute linearly to the overall time complexity, denoted as $\mathcal{O}(N)$. Thus, the effective time complexity of GraphFLEx is given by $\mathcal{O}(N + \mathcal{O}(\mathcal{M}_{gl}(\|S_i, X_\tau^i\|)))$. The overall complexity scales either linearly or sub-linearly, depending on α and the $\mathcal{M}_{gl}$ module. For instance, when $\mathcal{M}_{gl}$ is A -NN the complexity remains linear, if $\alpha \log(\alpha) \approx N$, whereas for $GLasso$, a linear behavior is observed when $\alpha^3 \approx N$.

H Node classification with GNNs and Graph Transformers.

We used four GNN models, namely GCN, GraphSage, GIN, and GAT. These GNN backbones (GCN, GraphSAGE, GIN, GAT) use two hidden layers of 64 units each and are trained for 500 epochs with Adam (learning rate 0.003, weight decay 5×10^{-4}).

We randomly split data in 60%, 20%, 20% for training-validation-test. The results for these models on Cora, Citeseer, and some synthetic datasets are presented in Table 11. Figure 7 illustrates the pipeline for training our GNN models. Graph structures were learned using both existing methods and GraphFlex, and GNN models were subsequently trained on both

Table 10. Unsupervised Graph Structure Learning Methods

Method	Time Complexity	Formulation
<i>GLasso</i>	$O(N^3)$	$\max_{\Theta} \log \det \Theta - \text{tr}(\hat{\Sigma}\Theta) - \rho \ \Theta\ _1$
<i>log-model</i>	$O(N^2)$	$\min_{W \in \mathcal{W}} \ W \circ Z\ _{1,1} - \alpha \mathbf{1}^T \log(W\mathbf{1}) + \frac{\beta}{2} \ W\ _F^2$
<i>l2-model</i>	$O(N^2)$	$\min_{W \in \mathcal{W}} \ W \circ Z\ _{1,1} + \alpha \ W\mathbf{1}\ ^2 + \alpha \ W\ _F^2 + \mathbf{1}\{\ W\ _{1,1} = n\}$
<i>large-model</i>	$O(N \log(N))$	$\min_{W \in \tilde{\mathcal{W}}} \ W \circ Z\ _{1,1} - \alpha \mathbf{1}^T \log(W\mathbf{1}) + \frac{\beta}{2} \ W\ _F^2$

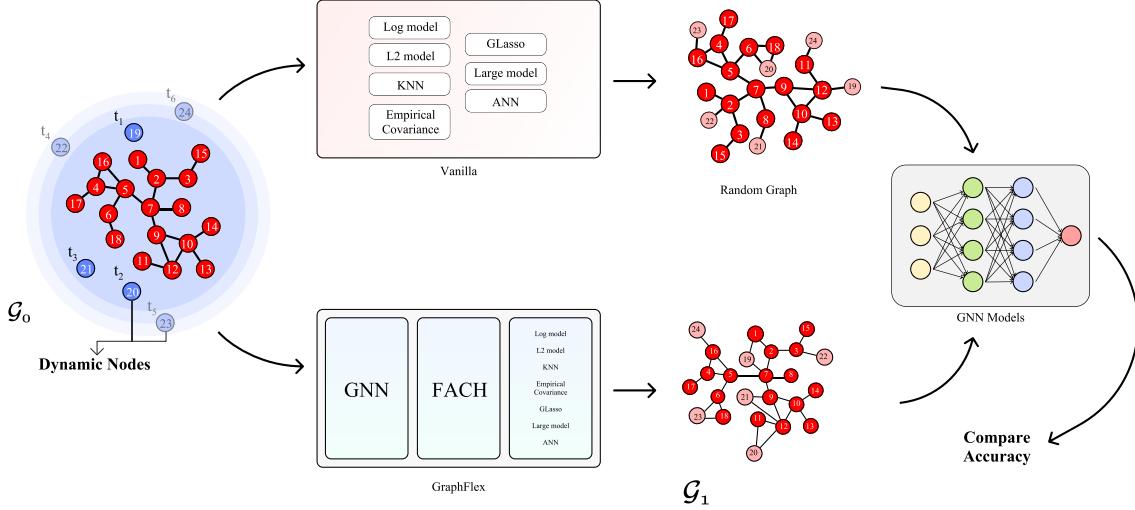


Figure 7. GNN training pipeline.

structures. Node classification accuracy results across all datasets are presented in Table 11 and Table 3. We further include node-classification accuracies of recent Graph Transformer models like GOAT (Kong et al., 2023), Nodeformer (Wu et al., 2022a), and Exoformer (Shirzad et al., 2023) when trained on graph structures learned by GraphFLEX (GLex) (see Table 12). Figure 8 shows the accuracies when structure is learned or given with 3 different scenarios (Vanilla, GraphFlex, Original) across different datasets.

I Computational Efficiency

Table 13 illustrates the remaining computational time for learning graph structures using GraphFLEX with existing Vanilla methods on Synthetic datasets. While traditional methods may be efficient for small graphs, GraphFLEX scales significantly better, excelling on large datasets like *Pubmed* and *Syn 5*, where most methods fail.

J Visualization of Growing graphs

This section helps us visualize the phases of our growing graphs. We have generated a synthetic graph of 60 nodes using PyGSP-Sensor and HE methods mentioned in Appendix A. We then added 40 new nodes denoted using black color in these existing graphs at four different timestamps. Figure 9 and Figure 10 shows the learned graph structure after each timestamp for two different Synthetic graphs.

K Ablation Study

In this section, we present an ablation study to analyze the role of individual modules within GraphFLEX and their influence on the final graph structure. Specifically, we focus on two aspects: (i) the significance of the clustering module, and (ii) the effect of varying module configurations on the learned graph topology.

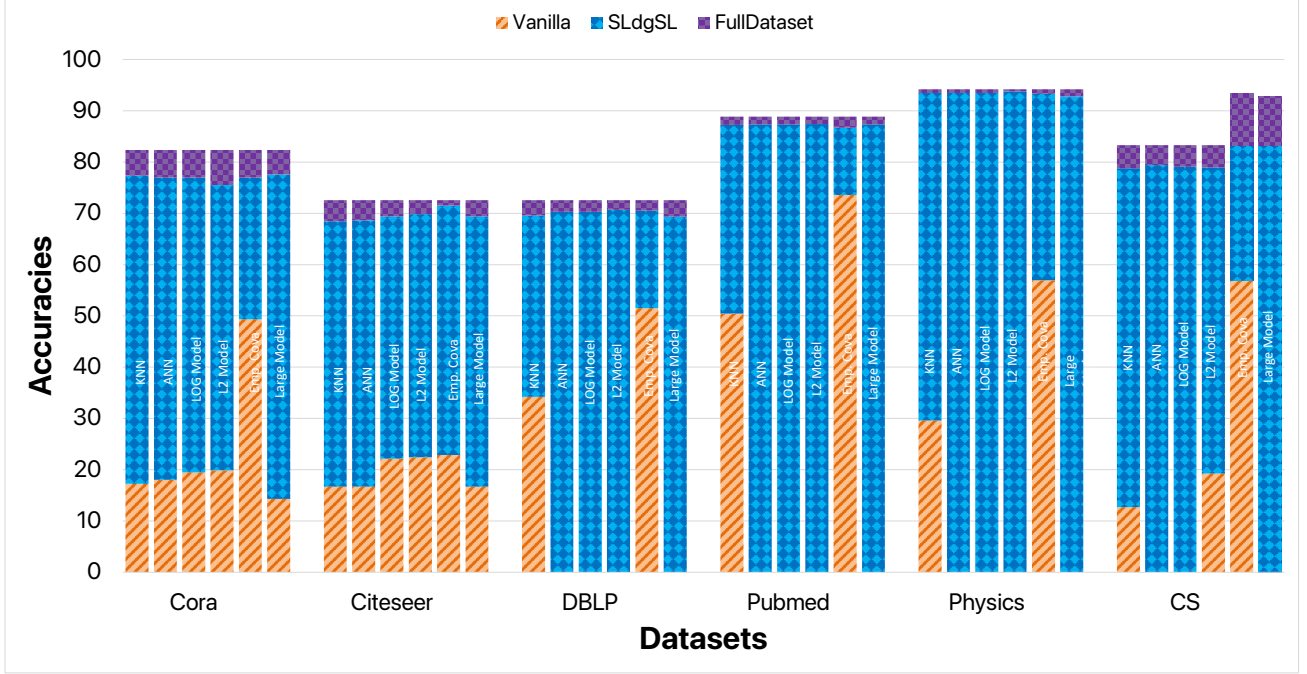


Figure 8. GraphSage accuracies when structure is learned or given with 3 different scenarios (Vanilla, GraphFlex, Original) across different datasets, highlighting performance with 30% node growth over 25 timestamps.

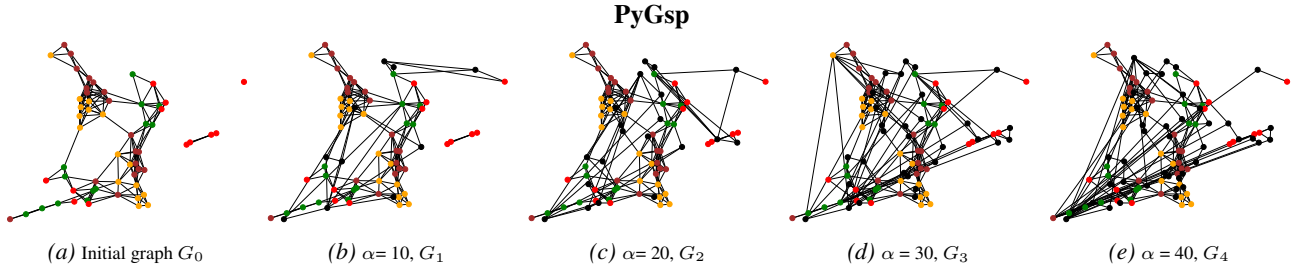


Figure 9. This figure illustrates the growing structure learned using GraphFlex for dynamic nodes. New nodes are denoted using black color, and α denotes number of new nodes. *PyGsp* denotes type synthetic graph.

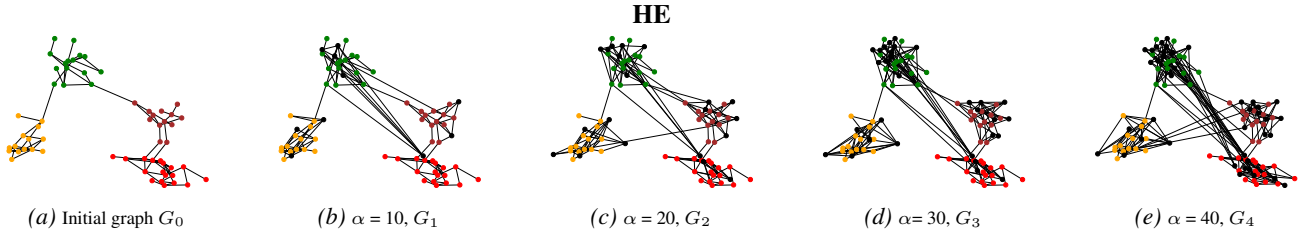


Figure 10. This figure illustrates the growing structure learned using GraphFlex for dynamic nodes. New nodes are denoted using black color, and α denotes the type of synthetic graph.

K.1 Clustering Quality

Figure 12 shows the PHATE (Moon et al., 2019) visualization of clusters learned using GraphFLEX’s clustering module $\mathcal{M}_{clust}$ for 6 single-cell RNA datasets, namely *Xin*, *MNIST*, *Baron – Human*, *Muraro*, *BaronMouse*, and *Segerstolpe* datasets.

Table 11. Node classification accuracies on different GNN models using GraphFLEX (GFlex) with existing Vanilla (Van.) methods. The experimental setup involves treating 70% of the data as static, while the remaining 30% of nodes are treated as new nodes coming in 25 different timestamps. The best and the second-best accuracies in each row are highlighted by dark and lighter shades of Green, respectively. GraphFLEX’s structure beats all of the vanilla structures for every dataset. OOM and OOT denotes out-of-memory and out-of-time respectively.

Dataset	Model	ANN		KNN		log-model		l2-model		COVAR		large-model		Base Struc.
		Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	Van.	GFlex	
Cora	GAT	18.73	73.84	20.96	73.65	16.14	72.36	18.74	73.10	49.72	77.55	14.28	76.43	79.77
	SAGE	17.25	77.37	18.00	76.99	19.48	77.40	19.85	75.51	49.35	76.99	14.28	77.55	82.37
	GCN	17.99	78.11	17.81	77.92	18.55	77.74	20.41	79.22	47.31	80.52	14.28	79.03	84.60
	GIN	16.69	76.44	18.74	80.52	17.44	76.25	19.29	76.62	48.79	78.85	14.28	76.06	81.63
Citeseer	GAT	16.51	61.82	25.00	62.27	19.24	64.70	18.18	63.48	20.91	62.73	16.67	62.27	66.42
	SAGE	16.66	68.48	16.67	68.64	22.12	69.39	22.42	69.85	22.88	71.52	16.67	69.39	72.57
	GCN	28.18	60.00	16.67	61.97	20.45	65.45	19.70	64.24	21.06	64.70	16.67	63.18	68.03
	GIN	16.66	64.39	16.67	63.94	20.15	59.85	18.64	63.64	22.12	60.30	16.67	61.81	67.38
Syn 4	GAT	29.55	92.07	OOM	90.86	OOT	91.64	OOT	91.64	35.79	92.52	OOT	93.74	89.49
	SAGE	26.75	87.89	OOM	91.05	OOT	86.64	OOT	86.64	32.92	90.44	OOT	86.01	90.03
	GCN	28.85	51.97	OOM	19.58	OOT	18.29	OOT	18.92	33.80	26.60	OOT	36.85	21.43
	GIN	28.50	65.61	OOM	31.06	OOT	26.51	OOT	26.56	34.03	46.40	OOT	47.10	29.35
Syn 6	GAT	44.00	86.80	43.60	86.60	30.00	78.75	55.40	92.80	36.20	93.60	31.80	92.80	97.20
	SAGE	41.00	93.80	41.40	93.60	33.75	88.75	57.60	94.00	35.20	94.80	28.20	95.60	97.40
	GCN	43.60	88.80	42.20	87.40	26.25	81.25	55.60	92.40	31.40	94.40	25.20	94.00	99.40
	GIN	39.60	89.00	40.40	86.60	21.25	82.50	55.20	91.80	30.00	94.60	30.40	92.00	98.80
Syn 8	GAT	29.55	99.75	33.75	88.75	88.25	99.25	88.25	99.25	26.00	85.50	94.00	96.00	98.50
	SAGE	26.75	100.0	32.50	100.0	88.75	99.50	88.75	99.50	26.75	100.0	92.50	100.0	100.0
	GCN	28.85	98.75	31.75	99.75	88.75	99.00	88.75	99.00	28.50	99.25	95.00	100.0	100.0
	GIN	28.50	50.00	30.50	91.00	82.25	91.50	82.25	91.50	27.25	81.75	91.75	92.25	78.25

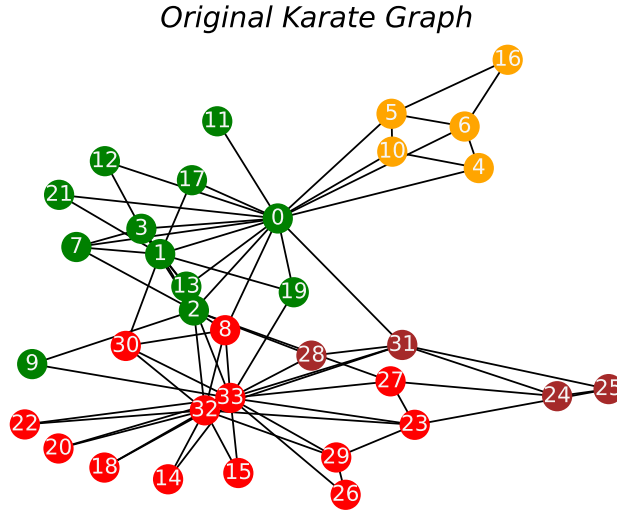


Figure 11. Original Karate Graph

K.2 Clustering Module Evaluation

To evaluate the effectiveness of the clustering module, we compute standard metrics such as Normalized Mutual Information (NMI), Conductance (C), and Modularity (Q) across various datasets (see Table 6 in Section 4.5). These metrics collectively

Table 12. Node classification accuracies on different recent Graph Transformer models when trained on structure learned using GraphFLEX (GFLEX). The experimental setup involves treating 70% of the data as static, while the remaining 30% of nodes are treated as new nodes coming in 25 different timestamps.

Data	Model	Base	GFLEX(KNN)	GFLEX(ANN)	GFLEX(Sub.)	GFLEX(covar)	GFLEX(l2)	GFLEX(log)
Cora	Goat	68.01	65.90	70.20	70.10	71.20	69.60	68.30
	Nodeformer	65.80	65.80	65.80	64.50	65.91	63.12	64.27
	Exoformer	71.80	68.16	69.80	70.30	67.70	69.80	70.30
Citeseer	Goat	65.02	65.10	62.40	66.13	61.20	63.03	66.10
	Nodeformer	61.00	61.00	61.71	63.48	62.60	61.34	61.54
	Exoformer	58.40	60.70	59.80	60.70	59.40	60.21	59.80
DBLP	Goat	81.92	80.68	80.31	82.25	82.29	79.86	80.10
	Nodeformer	73.76	73.76	75.34	74.39	72.94	73.29	74.96
	Exoformer	72.12	72.12	72.71	73.61	72.92	69.84	71.62
CS	Goat	93.05	93.24	93.32	90.84	92.39	93.16	92.03
	Nodeformer	94.60	95.12	95.12	95.36	94.91	95.12	93.29
	Exoformer	95.53	95.33	94.63	93.26	95.33	95.09	94.32

Table 13. Computational time for learning graph structures using GraphFLEX (GFLEX) with existing methods (Vanilla referred to as Van.). The experimental setup involves treating 50% of the data as static, while the remaining 50% of nodes are treated as incoming nodes arriving in 25 different timestamps. The best times are highlighted by color Green. OOM and OOT denote out-of-memory and out-of-time, respectively.

Data	ANN		KNN		log-model		l2-model		COVAR		large-model	
	Van.	GFLEX	Van.	GFLEX	Van.	GFLEX	Van.	GFLEX	Van.	GFLEX	Van.	GFLEX
Syn 1	19.4	9.8	2.5	10.5	2418	56.4	37.2	8.8	3.5	8.3	205	9.4
Syn 2	47.3	16.9	6.6	18.3	14000	144	214	22.6	20.3	18.6	1259	16.4
Syn 5	5.1	11.5	0.8	7.3	57.4	28	1.1	5.8	0.2	4.8	3.2	5.3
Syn 6	16.6	9.9	2.8	11.4	1766	96.3	193	101	5.3	8.9	324	9.6
Syn 7	10.6	7.4	1.4	8.9	704	85.2	10.3	7.9	0.9	6.4	36.5	8.2
Syn 8	19.6	11.2	2.5	11.7	2416	457	37.2	17.0	3.4	10.9	204	11.7

validate the quality of the discovered clusters, thereby justifying the use of a clustering module as a foundational step in GraphFLEX. Since clustering in GraphFLEX is applied only once on a randomly sampled small set of nodes, selecting the right method can be considered as part of hyperparameter tuning, where these clustering measures can guide the optimal choice based on dataset characteristics.

K.3 Impact of Module Choices on Learned Graph Structure

This section involves a comparison of the graph structure learned from GraphFLEX with existing methods. Six nodes were randomly selected and considered as new nodes. Figure 13 visually depicts the structures learned using GraphFLEX compared to other methods. It is evident from the figure that the structure known with GraphFLEX closely resembles the original graph structure. Figure 11 shows the original structure of Zachary’s karate club network (Zachary, 1977). We assumed six random nodes to be dynamic nodes, and the structure learned using GraphFLEX compared to existing methods is shown in Figure 13.

K.4 Sensitivity to Static Node Ratio

In the main experiments, we set the static node ratio to $r = 0.5$ and use $T = 25$ timestamps. This setting provides a balanced evaluation of both large-scale static graph learning and expanding graph learning, while keeping the candidate search space substantially smaller than learning on the full graph. However, smaller values of r further reduce the initial static graph size and therefore lower the computational cost, at the expense of having less initial structure for training the clustering module.

To evaluate this trade-off, we vary $r \in \{0.9, 0.7, 0.5, 0.3, 0.1\}$ and report node classification accuracy for different GNN

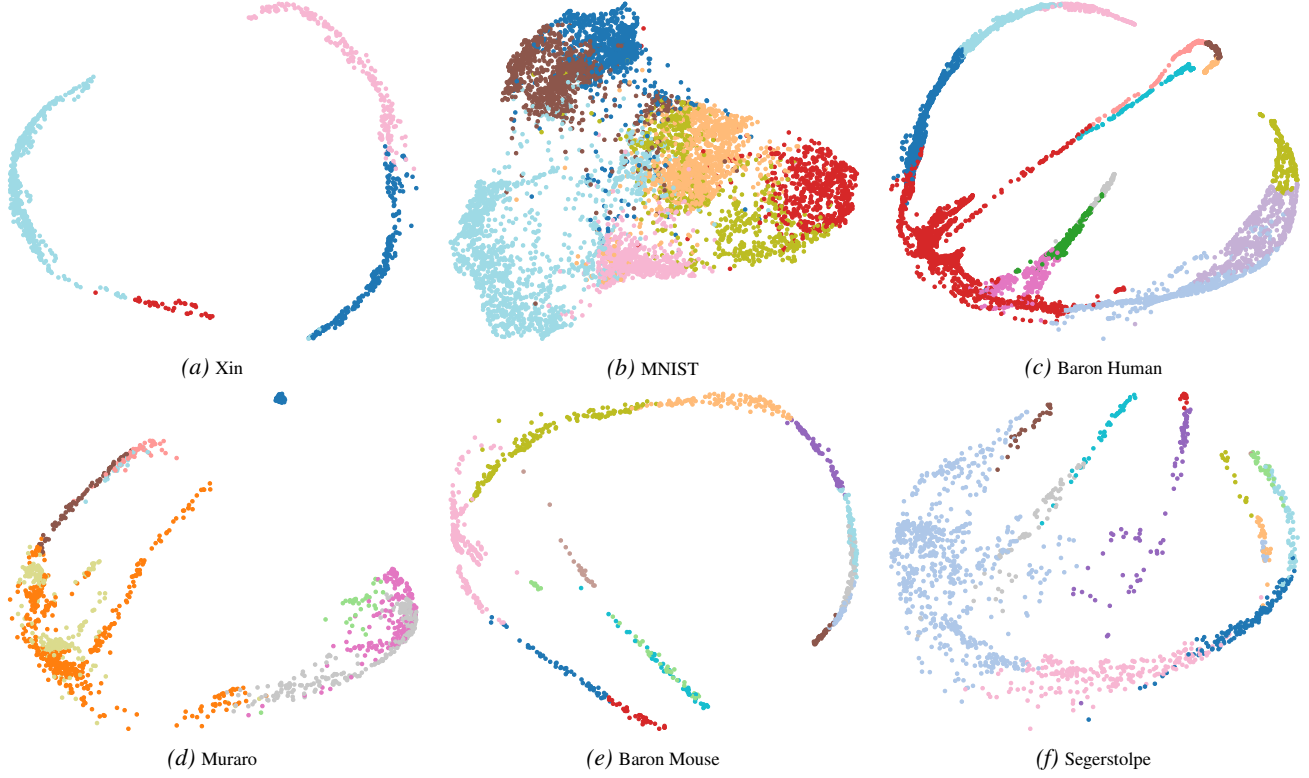


Figure 12. PHATE visualization of clusters learnt using GraphFlex clustering module for scRNA-seq datasets.

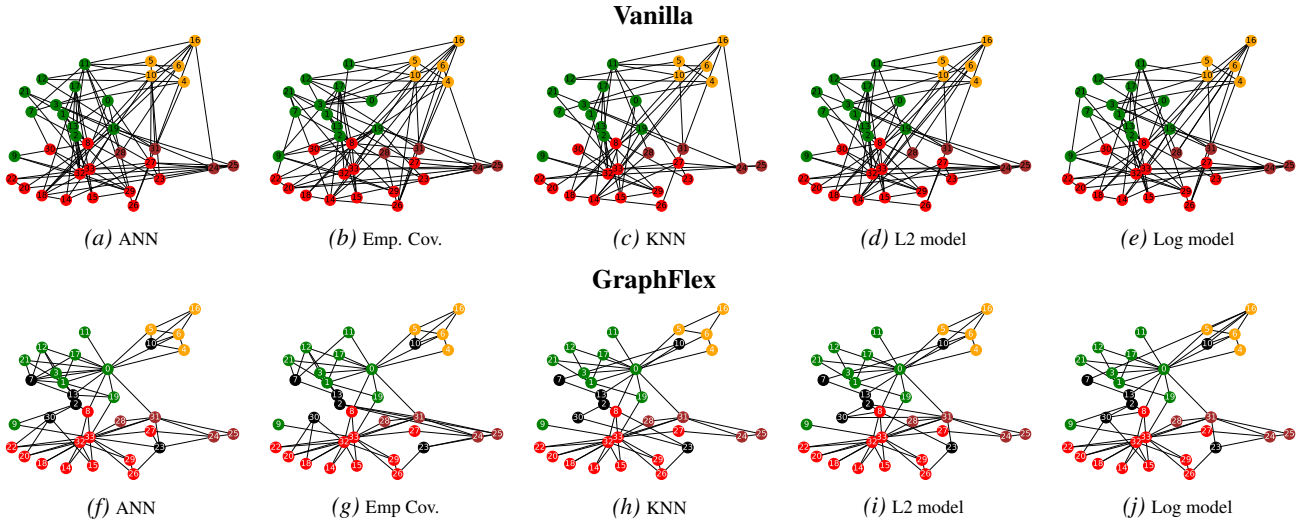


Figure 13. This figure compares the structures learned on Zachary's karate dataset when existing methods are employed with GraphFlex and when existing methods are used individually. We consider six nodes, denoted in black, as dynamic nodes.

backbones in Table 14. As expected, performance generally improves as r increases, since the initial graph contains more structural information. Nevertheless, GraphFLEX remains effective even for smaller values such as $r = 0.3$ and $r = 0.1$, showing that the framework does not critically depend on using half of the nodes as the static graph.

Table 14. Effect of static node ratio r on node classification accuracy.

Data	Model	$r = 0.9$	$r = 0.7$	$r = 0.5$	$r = 0.3$	$r = 0.1$
Citeseer	GAT	67.58	62.27	61.72	61.59	60.03
	SAGE	71.21	68.24	67.95	66.80	67.38
	GCN	65.78	61.97	61.30	60.93	60.51
	GIN	65.41	63.94	61.06	62.83	59.23
DBLP	GAT	69.37	69.30	69.31	67.05	62.69
	SAGE	71.72	70.39	70.07	69.78	65.15
	GCN	73.21	72.34	71.20	72.15	67.30
	GIN	70.88	67.19	68.97	65.94	63.11

These results show the expected accuracy–efficiency trade-off. Larger r provides more reliable initial structure and usually improves accuracy, while smaller r reduces the amount of data used in the static graph and therefore improves scalability.

K.5 Graph Classification

In graph classification datasets, each sample is typically a small graph, such as a molecule graphs (Hu et al., 2020). Therefore, applying clustering and coarsening to each individual graph is often unnecessary and may introduce avoidable overhead. We directly use the learning module to refine the graph structure and evaluate graph classification performance. We evaluate GraphFLEx on five graph classification datasets (Hu et al., 2020). Table 15 reports graph classification accuracy using GCN with different learning modules.

Table 15. Graph classification accuracy using GCN with GraphFLEx under different learning modules.

Dataset	Original	GFlex-KNN	GFlex-ANN	GFlex-Covar
MOLHIV	76.85	71.93	65.98	68.44
MOLTOX21	75.42	73.09	72.67	71.66
IMDB	73.00	54.00	55.00	50.00
REDDIT	78.50	74.50	76.50	63.07
PROTEIN	71.42	73.21	69.64	71.43

The results show that GraphFLEx extends naturally to graph classification by using its learning module directly. While the original graph structure remains competitive in several cases, GraphFLEx achieves comparable performance on multiple datasets. These results indicate that GraphFLEx is not restricted to node-level tasks, while also showing that clustering and coarsening are most beneficial for large graphs and are not always necessary for small graph-level datasets.

L Module Selection

GraphFLEx supports multiple GSL methods, see Figure 3. The Experiments section provides analyses to inform the selection of clustering and coarsening components. In particular, Section 4.5 evaluates clustering quality using quantitative metrics, which help choose a clustering method suited to a graph’s community structure and distribution. Moreover, Figs. 13 illustrate how different learning-module choices impact the final learned structure. Additional practical guidance is summarized below:

Clustering Methods. *K-means* is computationally efficient and performs well when clusters are roughly spherical; it is a strong choice for large datasets where speed is critical. *Spectral clustering* leverages eigenvalue decompositions to capture complex community structure, even when groups are not easily separated by simple distance metrics; however, it can be expensive at scale. *Deep learning–based clustering* adapts to non-linear, high-dimensional patterns and is effective for complex, feature-rich graph data, though it typically requires more compute.

Coarsening Methods. *UGC* (used in our main experiments) is a scalable, LSH-based approach that merges nodes with similar features and supports arbitrary coarsening ratios. Spectral approaches such as *LVE* and *LVN* preserve local eigenstructure but can limit coarsening flexibility. *Heavy Edge Matching (HE)* prioritizes the contraction of strong edges

and is efficient for edge-weighted graphs. *Algebraic-distance* and *affinity-based* schemes rely on node-proximity metrics and can be more costly. *Kron reduction* maintains spectral fidelity but is typically too slow for very large graphs. *FGC* jointly integrates graph structure and node attributes, optimizing clustering and coarsening together, but is computationally demanding.

L.1 Robustness to Clustering Quality

GraphFLEx mainly uses DMoN (Tsitsulin et al., 2023) as the default clustering module because it is directly motivated by modularity maximization and empirically provides strong community partitions. This aligns with our objective of identifying structurally meaningful node groups before applying coarsening and structure learning. However, GraphFLEx is not tied to DMoN; any clustering method can be used depending on the dataset and target graph property.

To study the sensitivity of GraphFLEx to imperfect clustering, we perturb the initial graph used for clustering by injecting Gaussian noise. We then evaluate both the clustering quality, measured using NMI, and the downstream node classification accuracy obtained after applying GraphFLEx.

Table 16. Effect of noisy clustering on downstream accuracy. Each entry reports NMI—GCN, SAGE accuracy.

Dataset	2%	5%	10%	15%	20%
CS	0.365—68.25, 75.18	0.058—66.38, 74.40	0.036—66.18, 74.08	0.022—65.69, 72.80	0.013—64.04, 71.80
DBLP	0.305—64.82, 67.54	0.212—64.66, 66.32	0.110—63.24, 65.90	0.058—60.14, 63.28	0.043—60.12, 62.10
Cora	0.168—72.17, 72.35	0.151—68.83, 69.16	0.152—67.04, 69.75	0.136—66.90, 68.60	0.121—64.75, 66.48
Citeseer	0.235—62.57, 67.42	0.013—67.27, 65.90	0.010—62.40, 66.08	0.010—63.36, 65.90	0.009—62.27, 65.90

We additionally vary the noise magnitude while perturbing 5% of the nodes. As expected, NMI decreases with stronger perturbations, indicating degradation in clustering quality.

Table 17. NMI under different Gaussian noise levels when 5% nodes are perturbed.

Dataset	$\sigma = 0.01$	$\sigma = 0.02$	$\sigma = 0.1$	$\sigma = 0.2$
DBLP	0.251	0.219	0.212	0.006
Cora	0.184	0.160	0.157	0.063
Citeseer	0.052	0.018	0.013	0.010
CS	0.323	0.099	0.058	0.014

Overall, while clustering quality affects performance, the downstream accuracy degrades only gradually under noisy cluster assignments. This suggests that GraphFLEx does not critically depend on perfect clustering. In practice, DMoN is a strong default choice, but the clustering module can be replaced based on dataset characteristics.

L.2 Sensitivity to Coarsening Ratio

The coarsening module controls the number of supernodes used during structure learning, and therefore introduces a natural trade-off between computational cost and structural resolution. A smaller coarsening ratio retains more nodes and provides finer structural information, while a larger coarsening ratio gives stronger compression and reduces the cost of graph learning. This also affects the candidate set used for connecting incoming nodes after projecting the learned coarse structure back to the original graph.

To study this trade-off, we vary the average coarsening ratio from 50% to 90% and report node classification accuracy using GCN and GraphSAGE in Table 18. The results show that GraphFLEx remains stable across a wide range of coarsening ratios. Accuracy decreases gradually as the graph is compressed more aggressively, with a more visible drop at very high coarsening ratios such as 90%, where fine-grained structural information may be lost.

Table 18. Effect of average coarsening ratio on node classification accuracy. Each entry reports GCN, GraphSAGE accuracy.

Avg. Coarsening	CS	Cora
50%	68.56, 75.10	73.41, 74.70
60%	67.54, 74.87	73.29, 74.61
70%	67.20, 74.75	72.10, 74.22
80%	66.53, 73.01	71.04, 71.92
90%	64.17, 72.30	68.26, 69.68

These results indicate that the coarsening ratio can be selected based on the desired accuracy–efficiency trade-off. Moderate coarsening ratios preserve sufficient structural information while substantially reducing the candidate search space. Extremely aggressive coarsening, however, may remove fine-grained neighborhoods required for accurate downstream prediction.

L.3 GraphFLEx as a synthesis of existing GSL approaches.

Many existing unsupervised GSL methods can be viewed as special cases of GraphFLEx that implicitly operate with a trivial clustering (single community), no coarsening, and a particular choice of M_{gl} (e.g. k -NN, graphical lasso, or GSP-based models). By making the clustering, neighborhood-preserving coarsening, and structure-learning components explicit and decoupled, GraphFLEx turns these implicit design choices into modular axes. This decomposition clarifies *where* scalability breaks down for vanilla methods (global vs. reduced node sets), and how different combinations of modules can be tailored to specific graph sizes, densities, and expansion dynamics.